Hochschule für Technik
und Wirtschaft Berlin
University of Applied Sciences

Named Entity Recognition in German-language Telegram channels

Bachelor Thesis

B.Sc.Computer Science and Business Administration

Faculty 4

by

Elisabeth Steffen

Berlin, August 19, 2022

1st Supervisor: Prof. Dr. Helena Mihaljević
2nd Supervisor: Prof. Dr. Theresa Busjahn

Abstract

This thesis investigates the potentials and limitations of Named Entity Recognition (NER) approaches for the automated analysis of text data from the messenger service Telegram.

The thematic focus is on the mobilizations for protests against the COVID-19 measures in Germany. The rise of conspiracy theories in the context of the COVID-19 pandemic not only reinforced antisemitic and racist discrimination against certain communities, but also generated an increasingly toxic and violent attitude towards political decision-makers and scientific experts.

The past months have once again shown the importance of investigative work by critical journalists and non-governmental organizations to critically monitor the corresponding mobilization strategies on Telegram. A central challenge here is the large amounts of user-generated data from often very dynamic contexts, which are usually analyzed manually. Here, approaches from the field of natural language processing can offer a valuable addition.

Against this background, this thesis aims to evaluate the applicability of Named Entity Recognition for the investigation of the COVID-19 protest movement in German-language Telegram channels, focusing on entities representing people, places, organizations, actions, dates, and times.

This work contributes to the development of German-language Named Entity Recognition in general by applying this approach to a specific text genre and domain, and provides first insights for developing an NER tool for researchers and non-governmental organizations in the future.

Contents

Chapter 1

Introduction

1.1 Background

The outbreak of the COVID-19 pandemic was followed by an increasing spread of misinformation and conspiracy theories. Those narratives evolved not only around the virus itself, but also about the intentions of those responsible for putting into effect measures for its containment. Those measures included the mandatory use of face masks, rapid testing, and vaccinations, at times functioning as admission requirements in the workplace or publicly accessible spaces such as restaurants and theatres. Furthermore, unprecedented restrictions were imposed on public life through several general or partial lock-downs of social and economic life, as well as temporary bans of gathering in public. These restrictions also heavily affected the possibilities to hold political demonstrations. From the beginning on, these measures were accompanied by critique and protests from various political spectres. In Germany, the so-called 'Querdenken' (lateral thinking) movement soon became the most prominent network organizing protests against the COVID-19 containment measures. The movement reached its preliminary climax with several large demonstrations taking place in Berlin, directed against legislative changes regarding the containment of the pandemic passed by the

German parliament. In the context of these demonstrations, violent clashes occurred between police and protesters, which often refused to act according the official restrictions such as wearing a face mask. Furthermore, there were several incidents in which journalists were violently attacked by protesters and accused of being part of the 'Lügenpresse' (lying press). Furthermore, there were two incidents in which the seat of the German parliament, the Reichstag, was targeted by protesters.

On April 21st 2021, the last large authorized demonstration of the 'Querdenken' movement took place in Berlin. In response to several violations of official restrictions by demonstration participants, police decided to disperse the demonstration. Several protesters subsequently moved to other areas nearby where clashes between protesters and police continued to occur throughout the rest of the day.

The subsequent demonstrations announced by the 'Querdenken' movement to take place in Berlin were prohibited by authorities, arguing with the experience at prior demonstrations that participants would act against against the official restrictions. In spite of the ban, several hundred protesters gathered in different spots in Berlin e.g. on the 21st, 28th and 29th of August 2021.

The mentioned bans of large demonstrations of the 'Querdenken' movement as well as the general restrictions for demonstrations and public gatherings (such as limiting the number of participants to a maximum of twenty persons), decreased the momentum of the large, centralized protests focussing on the German legislative located in Berlin. However, the protests did not stop as a result, but rather changed their character: They became more decentralized and a shift to more informal forms of gatherings and protests could be observed: In several German cities and towns, people gathered for 'Mahnwachen' or 'Spaziergänge' (walks) to protest against the government's COVID-19 politics. Since going for a walk individually was a legal outdoor activity even under the strictest lock-down regulations, this type of action

was chosen to circumvent the restrictions imposed on demonstrations. In effect, these walks ('Spaziergänge') often resulted in large gatherings of people moving through the streets together, often holding signs or chanting slogans against German politicians or the COVID-19 containment measures in general.

On some occasions, these walks turned into gatherings in front of the private homes of politicians in charge for the COVID-19 measures on federal state level: In December 2021, around thirty members of the far-right group 'Freie Sachsen' (Free Saxonians) gathered in front of the home of Saxony's Minister of Health, Petra Köpping, carrying torchlights. The incident received a lot of public attention, and was criticized in the media as well as by German politicians as attempts to intimidate politicians under the pretext of the protesters right to 'freedom of speech'. Earlier that year, the same group had gathered in front of Saxony's prime minister, Michael Kretschmer, who is one of the most prominent targets of the radical movement against the COVID-19 containment measures. In December 2021, an investigative report by German journalists revealed that members of the Telegram chat group 'Dresden Offlinevernetzung' had talked about concrete preparations to assassinate Kretschmer. And a few months later, by mid-April 2022, police foiled a plot to kidnap Germany's Minister of Health Karl Lauterbach and to sabotage power facilities in order to create chaos which would ultimately allow to overthrow the government. The rise of conspiracy theories in the context of the COVID-19 pandemic has thus not 'only' intensified hatespeech against and the discrimination of certain communities in the form of antisemitism and racism - this process was accompanied by an increasingly toxic and potentially violent climate for politicians in Germany.

The messenger service Telegram played a key role in the mobilization of the movement. The service allows the creation of large groups and channels to disseminate information quickly to a large number of users. Unlike

Facebook or Google, which hand over data to authorities at their request, Telegram does not cooperate with authorities. This ensures a high degree of anonymity for its users which has proven beneficial for protest movements in authoritarian countries, This anonymity poses a barrier to political repression and criminal prosecution. This makes the service attractive not only for social movements, but also for criminal and terrorist activities, and also facilitates the spread of hate-speech and calls for violence like those mentioned above.

The evolvement of the COVID-19 protests in Germany clearly demonstrate ow online and offline contexts influence and often reinforce each other. But even though non-governmental organizations such as e.g. CEMAS had warned about the antidemocratic and violent potential of parts of the movement, and had pointed out the involvement of political activists from Germany's extremist far-Right, German authorities underestimated the dangerous potential of this movement for a long time. Only after journalists had revealed the plans to murder Michael Kretschmer, and after several gatherings in front of private homes of politicians had taken place, the issue became more of a serious concern.

This points out the relevance of investigative work by critical journalists and non-governmental organizations to monitor the mobilization and networking strategies of movements such as 'Querdenken' in Germany. However, while these actors often have to deal with large amount of data, their investigation often relies on manually conducted research and data evaluation. Machine Learning-based approaches for Natural Language Processing (NLP) are often not easily accessible for these actors, making it hard for them to process large amounts of data from the often very dynamic contexts of informal political networks.

1.2 Objective

This thesis aims at exploring the potentials of a particular NLP technology, namely Named Entity Recognition (NER) for the investigation of the mobilization for protests against COVID-19 measures in a German-language Telegram channel. The overarching goal is to explore if a component for Named Entity Recognition could be a beneficial contribution to the Open Source project 'Teledash', to make NER available as a tool for journalists, NGOs, and researchers investigating the COVID-19 related protest movements and networks in Germany.

- approach of NER in German-language Telegram messages
- extraction of entities particularly relevant for mobilization / monitoring of potential targeting: PERSON, ORGANIZATION; DATE; TIME; LOCATION; ACTION as custom type
- explore potentials and limitations of existing models
- explore potentials of translation approach
- explore potentials of transfer learning with Transformer models

1.3 Thesis Structure

- Theory
- Exploration
- Implementation
- Conclusion and Discussion: suggestions for future work

Chapter 2

Theory

The first part of this chapter places Named Entity Recognition in its larger context of Natural Language Processing (NLP) in general, and of Information Extraction (IE) as a specific NLP task in particular. It provides a basic definition of of NER, lines out its historical evolvement, and discusses the challenges related to this task, both in general and with regard to the peculiarities of German language and to text data from social media.

The second part elaborates different approaches for NER, from knowledge- and rule-based methods over classical machine learning-based to deep learning-based approaches including transformers, and briefly discusses the limitations and potentials of the respective approaches.

2.1 Natural Language Processing

The field of Natural Language Processing combines approaches from computer science, linguistics, and artificial intelligence. Its main focus is to build systems and computational techniques for the automated processing and analysis of human language (cf. Vajjala et al. 2020, p. xvii).

Since its evolvement in the 1950s, NLP has been primarily an area of interest for academic research, but lately, it is being used more and more for

real-world tasks and business applications. (cf. Vajjala et al. 2020, p. xvii)

Core NLP applications comprise the use for email platforms (e.g. for spam filters), voice-based assistants, modern search engines, and machine translation services. (cf. ibid., p. 5). Furthermore, NLP is used for analyzing web and social media data (e.g. customer reviews), for spelling and grammar correction tools, and for a variety of domains including health care or finance. (cf. ibid., pp. 5–6).

Typical NLP tasks are for example language modeling, text classification, text summarization, information extraction, information retrieval, machine translation, or topic modeling (cf. ibid., pp. 6–7).

2.2 Information Extraction

Information Extraction (IE) is one of the several NLP tasks mentioned above. The aim of IE is to extract information from unstructured or semistructured natural language texts (cf. Zong, Xia, and Zhang 2021, p. 227).

'Information' can refer to different things, depending on the domain and purpose of the task: The extracted instances can be (named) entities such as people, locations, places, or organizations, but also important events occuring in texts (cf. Vajjala et al. 2020, p. 161). Sometimes the focus is on temporal information such as time and date, or on domain-specific entities, e.g. biomedical instances like genes, proteins, or viruses.

The extracted parts can also be attributes related to those entities, or relationships between the detected entities. The kinds of texts which information is extracted from may be news texts from the web, social media posts, customer reviews, academic literature, and others. The aim of IE is to generate structured output from this unstructured or semistructured text data. Typical tasks of IE are named entity recognition, entity disambiguation, relation extraction, and event extraction (cf. (Zong, Xia, and Zhang 2021, p. 227)

According to Zong, Xia, and Zhang 2021, IE technology can be classified via the domain aspect (limited vs. open domain), the type of extracted information (entity recognition, relation extraction, event extraction), and implementation methods (rule-based, machine learning-based, or deep learning-based approaches) (cf. ibid., p. 229).

Common examples of IE applications are news tracking, counter-terrorism, business and financial intelligence, the processing of biomedical data or scientific libraries. IE can also be utilized for applications building on entity-oriented search, for question-answering systems, or the task of text segmentation (cf. Aggarwal 2018).

The origins of IE dates back to the late 1970s, and its development was further accelerated in the late 1980s, when the US government financed activities to evaluate IE technology. Organized by the *Defense Advanced Research Projects Agency* (DARPA), the first *Message Understanding Conference* (MUC-1) took place in 1987. The conference provided standard datasets for international researchers to compete on and was held seven times between 1987 and 1997. The tasks included NER, coreference resolution, relation extraction, and slot filling, and mainly focused on text data from limited domains such as naval military intelligence, terrorist attacks, or aircraft crashes (cf. Zong, Xia, and Zhang 2021, p. 228). The first NER task was introduced at the Sixth Message Understanding Conference (MUC-6) in 1996. The aim was to apply information extraction technologies for largely domain independent named entity recognition. Besides recognizing the entity types person, organization, and location, the task also involved numerical expressions including time, currency, and percentage (cf. Grishman and Sundheim 1996). The corpus for the task consisted of 318 annotated articles from the Wall Street Journal. For the MUC-7 NER task, Chinchor developed annotation guidelines which were used as a basis for several later NER tasks (cf. Chinchor 1997.

By the end of the 1990s, the MUC was replaced by the *Automatic Con-*

tent Extraction (ACE) meeting which put its focus on extracting more fine-grained entities, as well as entity relations, and events. There was also a shift in the data to broader news data covering political and international events.

In 2009, ACE became a subtask of the *Text Analysis Conference Knowledge Base Population* (TAC KBP). This shifted the focus in data to open domain data (mainly from webpages), and the focus of tasks to entity attribute extraction and entity linking. Both MUC and ACE contributed to the development of several standard test datasets for the evaluation and development of IE approaches. (cf. Zong, Xia, and Zhang 2021, p. 229).

2.3 Named Entity Recognition

Named entity recognition is a subtask of IE and addresses the task of identifying named entities like person, location, organization, drug, time, clinical procedure, biological protein, etc. in text. NER is often used as the first step in question answering, information retrieval, co-reference resolution, topic modeling, etc. and is thus a fundamental task in NLP (cf. Yadav and Bethard 2019).

NER can be divided into the following two subtasks (cf.Zong, Xia, and Zhang 2021, p. 229):

- **1. Entity detection**, to detect whether a given string in a text document is an entity, including the detection of the entity's boundaries.
- **2. Entity classification**, to assign a specific category to the detected entity.

2.3.1 Common Entity Types

The entities which are to be identified via NER approaches are commonly represented by the following categories:

- PERSON
- LOCATION
- ORGANIZATION
- MISC/OTHER

Early NER approaches also included time, date, currency/quantity, and percentage as entity types. But while these can commonly be identified via regular expressions, instances of persons, locations, organizations, and miscellaneous/other usually do not follow a consistent pattern and thus pose larger challenges for NER, which is why most of NER research focusses mainly on these entity types (cf. Zong, Xia, and Zhang 2021, p. 229).

Also it should be noted that times, dates, currencies, quantities and percentage do not qualify as named entities in a narrow sense, but may be useful depending on the purpose of the application which NER is used for (cf. Aggarwal 2018, p. 386).

2.3.2 NER as a Sequence Labeling Task

NER is usually regarded as a sequence labeling task which means that to each word w_i in an input sequence, a label y_i is assigned, resulting in an output sequence Y which has the same length as the input sequence X (cf. Jurafsky and Martin 2021f, p. 1). Since named entities often consist of more than one token (e.g. Karl Lauterbach, Bill Gates Foundation, Straße des 17. Juni), named entity recognition is about labeling spans of texts instead of just single tokens. This makes it necessary to determine the correct boundaries of an entity in a text, and assign the correct label to each token in a detected span (cf. ibid., pp. 6–7).

2.3.3 The BIO Tagging Scheme

Since NER is usually regarded as a sequence labeling task, an appropriate label set as well as the language granularity for the annotation process needs to be defined for the respective sequence labeling model.

A widely used tagging scheme is the BIO label set (cf. Ramshaw and Marcus 1995). In this tagging scheme, the B-tag is used to label the beginning of an entity, the I-tag is used for labeling intermediate or final tokens of an entity, and the O-tag is assigned to all tokens outside of an entity.(cf. Zong, Xia, and Zhang 2021, p. 231).

Instances of different entity types representing persons, locations, or organizations are thus labeled with two different tags per entity type: B-PER, I-PER for persons, B-LOC, I-LOC for locations, and B-ORG, I-ORG for organizations (cf. ibid., pp. 231–232). The usual granularity levels for the annotation of entities are usually word- and character-level (cf. ibid., p. 232).

2.3.4 Evaluation of NER systems

Similar to other NLP tasks, NER involves an evaluation of the trained model. Evaluation can be done during model building, deployment, as well as in production, and rely on different metrics depending on the task at hand. Usually, the evaluation focuses on the model's performance on unseen data.

The evaluation during model building and deployment usually relies on intrinsic evaluation using ML metrics, while the evaluation in the production phase also involves application-specific business metrics, which is called extrinsic evaluation.

Extrinsic evaluation is usually way more expensive than intrinsic evaluation, and often requires the involvement of stakeholders, sometimes even end-users. **Intrinsic evaluation** using machine learning metrics on the other hand can be done by machine learning engineers themselves and is less cost-effective. Also, they are applied in the earlier phase of the process, thus

allowing early adjustments and quick improvments of the model (cf. Vajjala et al. 2020, p. 71). This is why intrinsic evaluation is usually used as a 'proxy for extrinsic evaluation.' ([71]ibid.).

To measure a model's performance by intrinsic evaluation, usually a test set is selected before training the model, and excluded from the training process. After training the model on the training set, the test set is used to evaluate the predictions of the model to assess its performance on unseen data. For NER, the manually annotated entity tags in the test set serve as gold reference with which the models predictions are compared (cf. Zong, Xia, and Zhang 2021, p. 241).

While intrinsic evaluation is a standard measure on most ML tasks, the specific metrics applied differ depending on the specific task.

Accuracy

Accuracy is a common machine learning metric which represents the percentage of correctly labeled named entities out of all tokens:

$$accuracy = \frac{TP + TN}{TP + FP + TN + FN} * 100$$

However, accuracy is not suitable for tasks such as NER, which are characterized by highly imbalanced classes. (cf. Jurafsky and Martin 2021a, pp. 65–66)In an annotated NER dataset, by far the most of the tokens will be assigned an O-tag, denoting they are not a named entity. Given a classifier which predicts this majority class for all tokens without having learned anything from the training data, this would still result in a high accuracy (e.g. an accuracy of 90% for a dataset containing 90% O-tags). This is why accuracy is not a suitable metric for NER. Instead, the metrics F1, precision, and recall are used.

F1, Precision, and Recall

Precision, **recall**, and **F1** score rely on the number of entities recognized correctly by the model (true positives, TP), the number of entities detected by the model, but not labeled as entities in the test set (false positives, FP), and the number of entities not recognized by the model, but tagged as named entities in the test set (false negatives, FN).

Precision, recall, and F1 are defined as follows:

$$precision = \frac{TP}{TP + FP} * 100$$

$$recall = \frac{TP}{TP + FN} * 100$$

$$F1 = \frac{2 * precision * recall}{precision + recall}$$

Precision thus represents the percentage of correctly identified named entities out of all labeled named entities, (i.e. the percentage of true positives out of all predictions), and recall equates to the percentage of named entities that are correctly recognized by the model out of all labeled named entities in the test set. The F1-score is the harmonic mean of precision and recall, and is the metric commonly used in shared tasks on NER to assess the performance of the participating systems (cf. Zong, Xia, and Zhang 2021, p. 241).

Evaluation of NER systems poses certain challenges compared to other tasks such as part-of-speech tagging: Since NEs can consist of multiple tokens, NER systems need to handle segmentation properly in order to achieve good results. This means that a system which labels only one token of a named entity consisting of two tokens correctly will cause two errors: A false positive (for labeling the second token as O), and a false negative (for not labeling the second token as part of an entity with the I tag) (cf. Jurafsky and Martin 2021f, p. 20).

Against this background, past shared tasks on NER have applied differ-

ent strategies to evaluate the participating systems: The MUC-6 task used (relaxed metrics), meaning that a prediction was considered as correct if the correct label was assigned, regardless of the correct boundaries, and that a prediction was correct if the correct text boundaries were detected, regardless of the correct label (cf. Yadav and Bethard 2019). The CoNLL tasks in 2002 and 2003 introduced **strict metrics**, meaning that a prediction was only considered as correct if both the predicted label and boundaries of an entity *exactly* matched the true label and boundaries (cf. ibid.).

2.3.5 Challenges for NER

Like other NLP tasks, NER is confronted with several challenges which stem from the characteristics of human language:

Ambiguity, understood as uncertainty or inexactness of meaning, is inherent to most human languages: Words or phrases can have multiple meanings, and usually it requires knowledge of the context in which they appear to resolve this ambiguity. Making computer-based systems take into account and 'understand' these contexts is one of the major challenges for NLP (cf. Vajjala et al. 2020, pp. 12–13) As for NLP tasks in general, ambiguity is also a core challenge for NER (cf. Aggarwal 2018, p. 386). For example, the word 'Bill' might refer to a persons first name and thus indicate a named entity of type PERSON, but when located at the beginning of the sentence, it might also be a synonym for 'invoice'.

For resolving this ambiguity, the **context** of a given word is usually of crucial relevance (cf. ibid., p. 387). NER systems differ in terms of how they can or cannot take into account the context of a word. Moreover, the data itself might not offer enough context. While articles from newspapers usually are of sufficient length and contain sufficient contextual information, social media texts are often of shorter length which results in a lack of context.

Common knowledge is often an essential part in a conversation between

humans: Facts are assumed to be known and thus not expressed explicitly in statements. Humans constantly use common knowledge to understand language. One of the challenges for NLP is thus how to encode this common knowledge for computational approaches (cf. Vajjala et al. 2020, pp. 13–14).

Creativity is another feature inherent to human language: Even though language is constituted via and essentially shaped by rules, it is far from being exclusively rule-driven. Rather, language is creative and heterogenuous; characterized by various styles, genres, and dialects. One important challenge for NLP is thus to enable machines to adequately handle this creativity (cf. ibid., p. 14). Moreover, language changes and evolves over time, in interaction with social and cultural processes, as well as with the development of communication technology (one example being the emergence of hashtags in the context of social media). NLP applications developed using data from the past might thus not fit well for data consisting of newly evolving manifestations of human language. These creative and dynamic characteristics of human language also pose challenges for the task of named entity recognition: Since new entities evolve over time, the set of know entities is never constant. While for 'some types of entities like locations, the names of entities evolve slowly, whereas for others like people and organizations, the incompleteness problem is very significant.' (Aggarwal 2018, p. 386).

Abbreviations referring to multiple instances of the same named entity are another important challenge (cf.ibid., p. 387). For example, the terms 'U.S.', 'USA' or 'United States of America' all refer to the same geopolitical entity. To overcome this challenge, usually entity disambiguation is an important step following named entity recognition.

Yet another challenge stems from **capitalization** conventions which vary across languages: While in English-language texts, capitalization usually indicates proper names and can thus be used as a feature for named entity recognition, in German-language texts all nouns are capitalized, making the feature less suitable (cf. Benikova, Biemann, and Reznicek 2014).

Diversity across languages makes it hard to apply an NLP solution from one language to another. According to the database *Ethnologue*, there are currently over 7.000 languages actively spoken worldwide (cf. Ethnologue 2022). Even though some of these languages share the same family and thus have some similarities, a direct mapping between two languages is not possible. Building language-agnostic solutions is a very complex matter, while building separate solutions for each language is labor- and time-intensive (cf. Vajjala et al. 2020, p. 14).

Imbalanced resources for different languages is yet another challenge for NLP, both in terms of research efforts and data. In fact, most NLP technologies are still designed for English or other European languages (cf. Singh 2008). For these so-called high-resource languages, a large number of datasets and libraries exists while low-resource languages are usually significantly 'less studied, resource scarce, less computerized' (Magueresse, Carles, and Heetderks 2020). This imbalance is also reflected in the existing NER datasets.

2.3.6 Specific Challenges for NER in User-Generated Texts

The aforementioned challenges for NER also apply for texts from social media platforms. However, named entity recognition for these user-generated texts can be regarded as even more challenging.

One reason for this is the challenge of **domain adaptation**: Most of the existing NER systems are trained on a handful of standard NER datasets which mainly consist of newswire texts. Since social media text data is often **more informal and noisy** (e.g. due to misspellings) than newspaper articles, which aggravates domain adaptation for this genre and usually results in poor performance of existing NER systems when applied to texts from social media (cf. Ritter et al. 2011).

Figure 2.1 illustrates the noisy character of tweets as an example.

1	The Hobbit has FINALLY started filming! I cannot wait!
2	Yess! Yess! Its official Nintendo announced today that they Will release the Nintendo 3DS in north America march 27 for $250
3	Government confirms blast n nuclear plants n japan...don't knw wht s gona happen nw...

Figure 2.1: Examples of noisy text in tweets, taken from Ritter et al. 2011

Furthermore, since texts from social media are often **shorter of length**, they lack sufficient context for the determination of an entity's type (cf. Ritter et al. 2011).

The **wide variety of capitalization styles** are another challenge, like e.g. the use of only lowercase for German texts, or the use of uppercase only in some texts (cf. ibid.). As ibid. point out, news-trained NER systems seem to rely heavily on capitalization which limits their applicability to user-generated social media texts.

Also, these texts usually contain more **out of vocabulary (OOV) words** than e.g. news articles. This is partly due to misspellings, but also stems from platform specific codes and styles: The frequent occurence of **special characters** such as # for hashtags or for user mentions poses challenges because they cannot easily be handled during annotation and training. User-generated text in social media also often includes a lot of **lexical variations**. As an example, ibid. collected more than fifty variations for the word 'tomorrow' from a Twitter corpus, as shown in figure 2.2.

‘2m’, ‘2ma’, ‘2mar’, ‘2mara’, ‘2maro’, ‘2marrow’, ‘2mor’, ‘2mora’, ‘2moro’, ‘2morow’, ‘2morr’, ‘2morro’, ‘2morrow’, ‘2moz’, ‘2mr’, ‘2mro’, ‘2mrrw’, ‘2mrw’, ‘2mw’, ‘tmmrw’, ‘tmo’, ‘tmoro’, ‘tmorrow’, ‘tmoz’, ‘tmr’, ‘tmro’, ‘tmrow’, ‘tmrrow’, ‘tmrrw’, ‘tmrw’, ‘tmrww’, ‘tmw’, ‘tomaro’, ‘tomarow’, ‘tomarro’, ‘tomarrow’, ‘tomm’, ‘tommarow’, ‘tommarrow’, ‘tommoro’, ‘tommorow’, ‘tommorrow’, ‘tommorw’, ‘tommrow’, ‘tomo’, ‘tomolo’, ‘tomoro’, ‘tomorow’, ‘tomorro’, ‘tomorrw’, ‘tomoz’, ‘tomrw’, ‘tomz’

Figure 2.2: Lexical variations of the word ‘tomorrow’ in tweets, taken from Ritter et al. 2011

Last but not least, social media texts often **mixed-language texts**, making it harder for language-specific models to accurately determine named entities in them.

Named entity recognition applications for other domains thus usually require the creation of custom corpora including a labor-intensive annotation process, or rely on automatically generated annotations of large text corpora, often resulting in inconsistent annotations.

2.3.7 Important Shared Tasks and Datasets for German-Language NER

The NER tasks at the Conference on Computational Natural Language Learning (ConLL) for English and German in 2003 was an important cornerstone for German-language NER (cf. Tjong Kim Sang and De Meulder

2003)

The provided German dataset consists of 909 articles sampled from the newspaper Frankfurter Rundschau (cf. Tjong Kim Sang and De Meulder 2003). The task focused on the entity types person (PER), location (LOC), organization (ORG), and miscellaneous (MISC).

The ConLL 2003 dataset is publicly accessible, but no licensing information is available. Furthermore, given the fact that the corpus only contains newspaper articles, its applicability to other domains might be limited. The annotation of the dataset was carried out at the University of Antwerp (cf. ibid.), thus most likely by non-native speakers, which might result in inconsistencies of the labeled data.

In 2014, GermEval, a series of shared tasks focussing on Natural Language Processing for German-language, announced a task for Named Entity Recognition for German-language texts (cf. Benikova, Biemann, and Reznicek 2014).

The provided dataset was compiled from German Wikipedia articles and online news. Similar to the ConLL 2003 task, entity types of interest were person (PER), organization (ORG), location (LOC), and other (OTH).

The dataset is publicly available under a e CC-BY license. However, since the GermEval2014 dataset also includes nested and derived entities, it is not readily combinable with other datasets. And even though the corpus was created from online resources, it does not contain noisy user-generated texts, limiting its applicability to other social media texts.

Up until today, the Germeval 2014 corpus and (to some extent) the ConLL 2003 dataset represent standards for German-language NER and are used to train and evaluate state-of-the art NER models.

2.3.8 Important Experimental Studies on NER on Social Media Texts

In 2011, Ritter et al. conducted an experimental study on named entity recognition in tweets, and used a random sample of 2.400 English-language tweets for this which were annotated with the following ten entity types: PERSON, GEO-LOCATION, COMPANY, PRODUCT, FACILITY, TV-SHOW, MOVIE, SPORTSTEAM, BAND, and OTHER (cf. Ritter et al. 2011). The best participating system in this task achieved an F1 score of 0.66 for these ten types (cf. ibid.).

A more detailed look into the results for the standard entity types PERSON, LOCATION, and ORGANIZATION, reveals that the F1 score for the entity type PERSON ranks highest with up to 0.83, followed by LOCATION, and then ORGANIZATION (0.74 and 0.66) for the best participating system.

In 2015, Derczynski et al. investigated the applicability of existing, by then state-of-the-art systems for named entity recognition in tweets. Their results indicate as well that existing machine-learning based approaches do not perform well on noisy user-generated data from microblogs, using three different English-language datasets, including the corpus created by ibid.

The evaluated systems achieved F1 scores ranging between 40.27, 51.49 (Ritter dataset), and 77.18 (cf. Derczynski et al. 2015).

In line with Ritter et al. 2011, the authors identify unreliable capitalization, typographic errors, shortenings, and lack of context due to the shortness of the texts as main challenges in this specific domain.

The authors suggest language detection, part-of-speech tagging, and normalization as pre-processing steps to encounter these challenges. Their results indicate however that e.g. normalization only leads to minimal increases, and in some cases even decreases the F1 score (cf. Derczynski et al. 2015).

2.3.9 Important Shared Tasks and Datasets for NER on Social Media Texts

Also in 2015, Baldwin et al. organized a shared task for Named Entity Recognition on social media data, namely tweets from Twitter in the context of the *Workshop on Noisy User-generated Text* (W-NUT). The task used the corpus created by Ritter et al. 2011.

Among the participating systems, the best F1 score achieved was 56.41, which is ∼0.1 lower than the F1 score achieved in the initial study by ibid., and ∼0.2 lower than the best F1 score achieved at the GermEval 2014 NER task (76.38, cf. Benikova et al. 2014).

The W-NUT series continued to address the task of NER in subsequent years: In 2016, another shared task on Twitter Named Entity Recognition was organized in the context of the workshop, again relying on the Ritter dataset and focussing on the ten entity types included in it. An important difference compared to 2015 was that some of the participating systems introduced the use of neural network methods for the task of NER (cf. Strauss et al. 2016), including the winning system which achieved an F1 score of 52.41 for the task of segmenting and categorizing entities into 10 types. It should be noted that the F1-scores for all other participating systems were below 50.00 for this task (cf.ibid.).

The W-NUT shared task in 2017 focused on novel and emerging entity recognition, thus addressing the challenge of newly emerging entities in social media data. The Ritter corpus was used as training data again. The development set consisting of ∼ 1.000 documents was composed of manually annotated texts from different online sources such as Reddit, StackExchange, and Twitter, while the test set (∼ 1.300 documents) consisted of manually annotated Youtube comments (cf. Derczynski et al. 2017). The annotation schema mapped the ten entity types of the Ritter corpus to the seven entity types person, location (including GPE, facility), corporation, product, creative-work, and group (representing music bands, sports teams, and non-

corporate organisations). The best participating system achieved an F1 score of 41.86. In the following years, the focus of the workshop series shifted to other tasks and domains which might be due to newly emerging research trends, but also due to the lack of relevant progress in terms of performance. Judging from the F1 scores of the respective winning systems, the task of NER in social media texts continued to be a considerable challenge. Furthermore, the ongoing shortage of manually annotated and sufficiently large corpora for this task becomes apparent. This shortage has been tackled by the creation of human-annotated NER corpora such as the Broad Twitter Corpus (cf. Derczynski, Bontcheva, and Roberts 2016, which is unfortunately exclusively English-language.

However, the results presented so far might also indicate that by then state-of-the-art machine learning systems had reached their limits. The following section traces the evolvement of NER approaches over time and gives insights into the respective potentials and limitations.

2.4 Overview of NER approaches

Since it's beginnings in the 1970s, NER approaches have undergone several phases of development, parallel to the advancements in NLP technologies: Starting with early rule-based and knowledge-based systems, the field moved on to machine-learning approaches, including probabilistic models and log-linear models based on feature-engineering. Later on, neural networks, using word embeddings and only minimal feature-engineering, marked another important step forward in the field. Lately, the introduction of transformer models, achieve new state of the art performance for several NLP tasks, marked another crucial advancement for NER.

2.4.1 Rule-based and Knowledge-Based Approaches

The first systems for NER relied on handcrafted rules, lexicons, orthographic features and ontologies (cf. Yadav and Bethard 2019).

Rule-based systems make use of the fact that the structure and the context of named instances of type person, location, and organization often follow certain rules and can thus be recognized to some extent by using e.g. regular expressions (cf. Zong, Xia, and Zhang 2021, p. 230). Hand-crafted rules can make use of features such as for example capitalization (used in a lot of languages for the beginning of person names), or certain 'key words' such as titles for persons (Mr., Mrs. Dr., etc.) (cf. ibid., p. 230).

However, these approaches are limited even when a large amount of data is considered. This is due to the ambiguity of language, resulting in different meanings for a given word or sequence in different contexts. Also, these approaches are rather static and often struggle with handling newly emerging identities without updating the underlying rules. Furthermore, the generation of rules is labor-intensive and usually requires expert domain knowledge. (cf. ibid., p. 231).

Knowledge-based approaches rely on lexicon resources and domain specific knowledge. The performance of these systems heavily depends on the exhaustiveness and completeness of the used lexicon. Here as well, the process can be rather labor-intensive because domain expert knowledge is needed for the construction and maintenance of the knowledge resources (cf.Yadav and Bethard 2019).

2.4.2 Classic Machine Learning-Based Approaches

Machine-learning based approaches introduced a critical advancement for the task of NER since they do not require the manual creation of hand-crafted rules or exhaustive lexicons (cf. Nadeau and Sekine 2007).

These approaches instead focus on the development of algorithms which

are able to perform a given task by automatically learning from a large number of examples, the training data (cf. Vajjala et al. 2020, p. 14).

The process usually involves the creation of a numeric representation of the training data, called features. These features are then processed by the model which is supposed to find patterns in the training data (cf. ibid., p. 15). The introduction of machine learning models thus allowed for the development of NER approaches based on an automated learning process.

Machine learning algorithms can roughly be grouped into three paradigms: supervised learning, unsupervised learning, and reinforcement learning. For the sake of brevity, I will focus on examples of supervised learning and not go into further detail about the latter two concepts.

In supervised learning, the machine learning model learns to make predictions for unseen data from a large number of labeled examples, called training data. The training data usually consists of input-output pairs. For NER, an input example would be a text containing different instances of entities, and the corresponding output would then be corresponding entity tags and spans which determine the type and the boundaries of each named entity in the text example.These outputs are also called labels or ground truth. In order to train a well-performing model which is able to predict the spans and tags for entities in unseen texts, large amounts of labeled data are required (cf. ibid., p. 15).

Classic algorithms used for NER are for example Hidden Markov Models (HMM), Maximum Entropy Models (MEM), Support Vector Machines (SVM), Decision Trees, and Conditional Random Fields (CRF) (cf. Yadav and Bethard 2019).

Generative models like HMMs are modelled in a way that they would be able to generate an example of a given class, while discriminative models like CRFs focus on discriminating between possible classes by focussing on distinct features of each class, without necessarily learning much in general about the class itself (cf. Jurafsky and Martin 2021c, p. 77).

I will explore HMMs and CRFs as two examples for classic machine learning based approaches in more detail in the following section.

2.4.3 Hidden Markov Models

An HMM is a probabilistic model which, when applied for sequence labeling tasks, finds the most likely sequence of labels for a given input sequence of words. It does so by computing a probability distribution over possible label sequences, and then chooses the best (i.e. the most likely) sequence of labels for the given input sequence. (cf. Jurafsky and Martin 2021f, p. 8).

HMMs are based on the assumption that there is an underlying structure, called hidden states (e.g. the grammar of a language), which generates the observed states (the actual words appearing in a sentence in a specific order). (cf. ibid., p. 9) This assumption reflects the fact that when dealing with language, we often cannot directly observe events. For NER, the events we can directly observe are the actual words in a sentence, while the corresponding NE tags need to be inferred from these words, and are thus hidden events.

HMMs can thus be understood as state machines modelling different states (represented e.g. by NE tags) as nodes, and transitions between these states as edges. (cf. ibid., p. 8). The transitions between the states are associated with different probabilities, which represents the fact that some tags are more likely to follow a given tag than others.

For example, if the current token is labeled with an O tag, the probability for the next token's label to be an I tag would be zero (since an entity span always begins with a B tag and can never begin with an I tag), while it could be equally likely that the next token is the beginning of an entity (B tag) or still outside of an entity (O tag).

HMMs are furthermore based on the Markov assumption, which means that the probability of a particular state q_i depends only on the previous state $q_i - 1$, and not on any other previous states (cf. ibid., p. 8). For NER this means that the context considered for predicting an NE tag for the current

token is limited to the tag of the preceding token.

This means for example that if the current tag is an I tag, the model knows only the previous tag, which is an I tag. But it does not know how many tokens before the entity has started (a token represented by a B tag) and thus cannot take this into consideration for predicting the next tag.

An HMM thus models a) the probabilities for transitions between hidden states (NE tags), and b) the probabilities for observations (actual words) associated with each of these states. Figure 2.3 illustrates an HMM for part-of-speech tagging which works similar to NER.

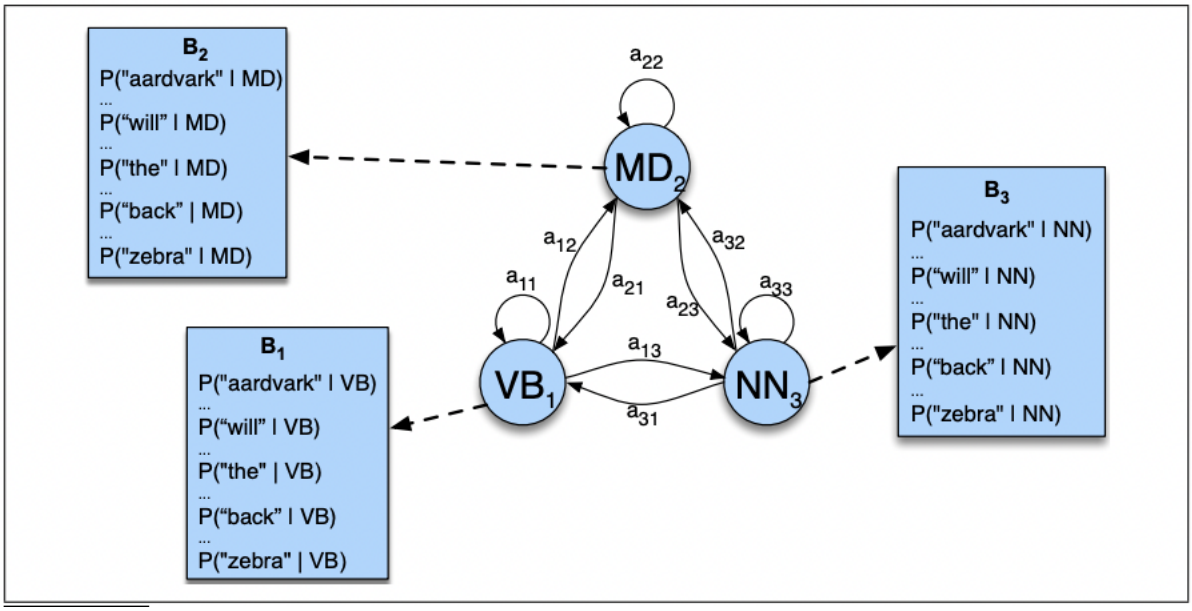

Figure 2.3: An illustration of an HMM for part-of-speech tagging; modelling transition probabilities between POS tags and observation likelihoods for words occuring with these tags. Taken from Jurafsky and Martin 2021f, p. 11

The illustration clarifies that the probabilities computed by the HMM tagger are a) based on the assumption that the probability for a word to appear depends only on the word's tag, and b) that the probability for a tag to appear depends only on the tag which appeared directly before the tag (cf. Jurafsky and Martin 2021f, p. 11).

The task of the HMM tagger is thus to find the most likely sequence of tags (hidden states) for a given input sequence of observations (words). This task of determining hidden variables for a sequence of observations is called decoding (cf. Jurafsky and Martin 2021f, p. 11). For decoding in HMMs, the Viterbi algorithm is used to find the state path which assigns the maximum likelihood for a given observation sequence (cf. ibid., p. 12)

Since human language is inherently sequential, the assumption embodied in HMMs that the current word depends on the word occuring before it is makes these models powerful tools for modeling textual data (cf. Vajjala et al. 2020, pp. 21–22).

However, a limitation of these models is their restricted capability to integrate arbitrary features, e.g. to take into account capitalization or morphology of words. These kinds of features are important though when dealing with unknown words (cf. Jurafsky and Martin 2021f, p. 15), which are likely to appear in most NLP tasks due to the creative and dynamic character of human language.

In order to incorporate such arbitrary features, discriminative models such as as Conditional Random Fields (CRF) are more suitable than generative models such as HMMs.

2.4.4 Conditional Random Fields

CRFs are log-linear models, which means that they rely on logistic regression. Logistic regression is a fundamental algorithm for classification in supervised machine learning, and also closely related with neural networks which I will describe later. Therefore I will now briefly describe the very basics of logistic regression before I go into detail about CRFs. For a detailed explanation of logistic regression see Jurafsky and Martin 2021c.

Logistic Regression

Logistic regression is a supervised machine learning algorithm which can be used for classification tasks, which can be either binary (two different classes) or multinomial (multiple classes) (cf. Jurafsky and Martin 2021c, p. 1).

Like other supervised machine learning algorithms, logistic regression requires a training corpus consisting of inputs and outputs, e.g. text and the corresponding labels.

A logistic regression classifier consists of the following components (cf. ibid., p. 2):

- A numerical feature representation for each input in the form of a vector $[x_1, x_2, ..., x_n]$.
- A classification function, e.g. the softmax function, which takes care of computing the probability for a given input to belong to one of the provided classes.
- An objective function for learning, e.g. the cross.entropy loss function, which is designed to minimize the error of the model when trained on the provided training examples.
- And an algorithm which is used to optimize the objective function, e.g. the stochastic gradient descent.

During the training phase, the model learns a weight for each feature and a bias term b. These parameters are usually randomly initialized at the beginning of the training, and then iteratively adjusted. The adjustment (i.e. learning) is carried out using the loss function which computes the error between the model's prediction and the true label of a given input, and the stochastic gradient descent, which takes care of updating the weights and the bias term so that the loss is minimized. The weights represent how important the respective feature is for the classification decision ((cf. ibid., p. 3)

The task is thus to learn parameters from the training data which allow the model to provide accurate predictions also for unseen data. Thus usually a part of the labeled data is excluded from training, and then used as a test set for evaluating the model. Commonly, this test set is further split up into a test and validation set. This allows to use the validation set for evaluation and hyperparameter optimization during training, and excluding the test split for final evaluation.

The probability is computed by multiplying each feature x_i with its learned weight w_i, then summing up these weighted features, adding the bias term b, and then passing the outcome to the classification function to transform the intermediate, real-valued result into a probability ((cf. Jurafsky and Martin 2021c, pp. 2–3).

Now that we have learned the very basics of logistic regression, we can turn to CRFs again. A CRF is a log-linear model which gets an input (word) sequence X as input, and finds the best output sequence Y by comparing its probability to all all possible tag sequences $\mathscr{Y}$ (cf. Jurafsky and Martin 2021f, p. 16).

While an HMM computes a probability for each tag per time step, CRFs compute log-linear functions over a set of features, called local features, at each time step. For computing the local features, the CRF makes use of the current output token y_i, the previous output token $y_i - 1$, the entire input string X, and the current position i (cf. ibid., p. 16). Each feature is assigned a weight, and a function F maps the input sequence and the output sequence to a global feature vector $F_k(X, Y)$. These local features are then aggregated and normalized, resulting in a global probability for the whole sequence (cf. ibid., p. 16).

The limitation to the previous output token at $i - 1$ is a characteristic for linear chain CRFs which are most commonly used for NLP tasks. This restriction allows the use of the Viterbi algorithm for decoding. While general CRFs would allow to consider more context (i.e. more distant tokens), this

makes the decoding process more complex, which is why general CRFs are less commonly used for language processing tasks (cf. Jurafsky and Martin 2021f, p. 16).

Since CRFs allow the incorporation of arbitrary features, the task of feature-engineering becomes central for the modelling process. The decision which features are to use is taken by the machine learning engineer, but the population of the features is done automatically using feature templates (cf. ibid., p. 16). Features for an NE task would of course be the actual word and the corresponding label, e.g. $x_i = Karl$ and $y_i = B - PER$. Furthermore, to handle unknown words, word shape features can be added to create an abstracted representation of the word which maps lower-case letters, upper-case letters, and numbers to x, X, and d, and retains the punctuation of the word, which would be $word\_shape(x_i) = Xxxx$ for our example. Yet another feature could model which part-of-speech the current word represents, which would be proper noun for the given example. Gazetteers are also often used for NER feature-engineering: These are large indices containing millions of location names. The corresponding feature would indicate whether a given word appears in a given gazetteer, making it especially useful for the recognition of named entities of type location (cf. ibid., p. 18). Other resources can be name lists to indicate whether a given word represents a person's name.

Figure 2.4 shows an example sentence with some NER features as they could be used in a CRF.

Words	POS	Short shape	Gazetteer	BIO Label
Jane	NNP	Xx	0	B-PER
Villanueva	NNP	Xx	1	I-PER
of	IN	x	0	O
United	NNP	Xx	0	B-ORG
Airlines	NNP	Xx	0	I-ORG
Holding	NNP	Xx	0	I-ORG
discussed	VBD	x	0	O
the	DT	x	0	O
Chicago	NNP	Xx	1	B-LOC
route	NN	x	0	O
.	.	.	0	O

Figure 2.4: NER features for an example sentence, taken from Jurafsky and Martin 2021f, p. 19

The example makes clear that CRFs differ from HMMs not only with regard to the underlying algorithmic paradigm (generative vs. discriminative, probabilistic vs. log-linear models), but also in terms of how rich feature representation can be. However, this richness of features comes at the price of often labor-intensive feature engineering. Furthermore, we can see that these approaches also partly rely on knowledge sources like e.g. gazetteers or name lists, making them dependent on the exhaustiveness of such resources, resulting in limitations for the recognition of newly emerging entities.

While HMMs were commonly used for NER in the late 1990s, the use CRFs became more and more prominent by the beginning of the 2000s, accompanied by a phase of intense feature engineering and exploration (cf. Jurafsky and Martin 2021f, p. 24). These feature-based NER systems however have some disadvantages: First, Feature-engineering is a time-consuming part of system design. Second, inputs in these systems are usually represented as sparse vectors, which limits the possibilities to model semantic word similarity or synonymy. The introduction of embeddings, representing inputs as dense vectors, thus marked another important step forward. In

combination with neural networks, they marked the next important step in the evolvement of NER (and of NLP in general).

2.4.5 Neural Networks and Word Embeddings

NER systems using deep neural networks have been introduced in the first decade of the 2000s.

Neural networks consists of a number of small computing units. Usually, these networks consist of multiple layers, which is why they are called deep neural networks. The very basic principle of a neural network is that each of its units takes a vector as an input and produces an output value. For computing this output value, the unit multiplies the input vector by a weight matrix, adds a bias term, and passes the sum of all values to a so called activation function which produces a representation of the given input. This representation can either be the input for the next hidden layer of the network, or be processed by the output layer to compute the final output of the model, e.g. for classification tasks (cf. Jurafsky and Martin 2021e, pp. 128–137).

Collobert and Weston 2008 presented one of the first neural network architectures for NER. The basic idea was to use a unified neural network architecture to train a general language model, and then fine-tune this model using labeled training data for specific NLP tasks.

While Collobert and Weston (2008) still involved minimal feature-engineering (via using capitalization for the creation of the feature representations), later approaches replaced these manually constructed feature vectors with word embeddings: In 2011, Collobert et al. proposed their pioneering approach based on a 'unified neural network architecture and learning algorithm that can be applied to various natural language processing tasks including part-of-speech tagging, chunking, named entity recognition, and semantic role labeling.' (Collobert et al. 2011). The prior paragdim to rely on 'man-made features carefully optimized for each task' (ibid.) was now replaced by the

use of 'internal representations [learned by the system] on the basis of vast amounts of mostly unlabeled training data.' (Collobert et al. 2011).

Word representations are thus no longer based on handcrafted features in this approach. Rather, the inputs were only minimally pre-processed and then fed to a neural network with multiple layers in which word embeddings were created which were then used for task specific fine-tuning.(cf. ibid.).

Word embeddings are representations of words in n-dimensional space which are typically learned automatically using large amounts of unlabeled data, for example using the skip-gram model introduced by Mikolov et al. 2013. Earlier machine approaches for NLP typically treated words as atomic units, and represented them via their index in a vocabulary (cf. ibid.), or, depending on the task, via the various handcrafted features as we have seen above, e.g. modelling the information whether a given token is included in a given gazetter as 0 or 1. The results are long, sparse vectors on which the model is trained. However, this form of representation does not easily allow to embody a notion of similarity (or synonymy) between different words. Word embeddings by contrast are short, dense vectors, constructed from large amounts of data, to represent words in a way that captures semantic similarity, resulting in a semantically more expressive representation of words (cf. Jurafsky and Martin 2021d, p. 17)

The first NER approaches using neural networks and word embeddings relied on static embeddings. Static means that each word in the vocabulary is represented by a fixed embedding, regardless of the different contexts in which it might appear (cf. ibid., p. 18). While I focused on word embeddings here as an example, it should be noted that neural networks for NER can and have also been trained on embeddings generated on character or subword level (cf. Yadav and Bethard 2019).

The NER approaches from this phase thus differ fundamentally from prior work in the way the inputs are represented. Yet they also differ in terms of the underlying architecture: Making use of neural networks opened

up new and more elaborate possibilities for considering the context of a given token. Against this background, neural network architectures such as recurrent neural networks (RNNs) and long short-term emory (LSTM) networks were increasingly used for NER.

2.4.6 Recurrent Neural Networks

Recurrent neural networks (RNNs) are a specific type of neural networks which are designed to process input from prior layers in the network recursively. The neural units in RNNs are designed in a way that allows them to 'remember' what they have processed so far (cf. Vajjala et al. 2020, p. 23). This is accomplished by creating a recurrent link between the current layer and the preceding layer. The input from the preceding layer provides information which can be conceptualized as memory, or context. This context is not limited in terms of length, meaning that the 'memory' can go back to the very beginning of the sequence (cf. Jurafsky and Martin 2021g, p. 4).

However, in practice the incorporation of distant information is difficult in RNNs. Even though the model has access to the entire preceding context, the encoded information tends to represent the more recent parts of a sequence (cf. ibid., p. 13). Furthermore, RNNs struggle with challenges during training due to the so-called vanishing gradients problem which I will not explore in detail here. As a consequence, RNNs do not perform well with long input sequences ((cf. ibid., p. 15). Furthermore it means that in practice, they are not able to represent information distant from the current processing point (cf. ibid., p. 13. Long short-term memory (LSTM) networks are designed to solve this issue.

2.4.7 Long Short-Term Memory Networks

LSTMs are neural networks which handle context by adding different gates to the network. These gates are used to remove information which is no

longer needed, and to add information which is likely to be useful for subsequent processing steps and the final decision-making (cf. Jurafsky and Martin 2021g, p. 15). Accordingly, these gates are called **forget gate** and **add gate**. They are used to create a context vector c_t as an output for each layer of the network. At the end of each LSTM unit exists an **output gate** which determines which information is used to produce the hidden state h_t as the second output of the layer. The inputs for each layer are the current input x, the previous hidden state $h_t - 1$, and the previous context $c_t - 1$ (cf. ibid., p. 16). The basic concept of LSTMs is thus to take context into account in a more sophisticated way: By forgetting information considered irrelevant, and remembering information considered relevant for the final classification decision. Thus, LSTMs offer a more effective way to process longer inputs, since the information load is reduced by only selectively passing contextual information on to the next layer (cf. Vajjala et al. 2020, p. 23) LSTMs used for NLP are often bidirectional (Bi-LSTMs) which means that they process information from both the past (e.g. the words occuring before the current word), and the future (the words occuring after it). This is usually done by combining two independent networks, one of which processes the input from left to right, and the other one from right to left (cf. Jurafsky and Martin 2021g, p. 13). Such BiLSTMs are furthermore often combined with CRFs, such as in the approach proposed by Huang, Xu, and Yu 2015, who used this kind of architecture for sequence tagging, achieving an F1-score of 88.83 and thus slightly outperforming the above-mentioned work of Collobert et al. 2011 (F1=88.67). Even though LSTM manage to handle more distant information than RNNs, both architectures still suffer from information loss and difficulties in training. Moreover, both follow a sequential design pattern, processing one input after another. This results in limitations when it comes to parallelization, which is important to process larger amounts of data fast and efficiently (cf. [17]Jurafsky and Martin 2021g. Transformer models represent an approach which allows for parallelization by dispensing with the

concept of recurrency, replacing it with a mechanism called self-attention.

2.4.8 Transformer Models

In recent years, transformer models have achieved state of the art results for a variety of NLP tasks and represent the latest advancement in deep learning architectures for NLP.

The architecture was first introduced in 2017 by Vaswani et al., in their seminal paper 'Attention Is All You Need', in which the authors propose a 'new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely.' (Vaswani et al. 2017)

Transformers thus represent a clear shift from prior model architectures. Like the architectures described above, transformer models map input sequences to output sequences of the same length. This sequence-to-sequence architecture is also called encoder-decoder architecture (cf. Tunstall et al. 2022, p. 3).

But unlike prior approaches, transformers do not follow a sequential pattern for modelling context. Rather, they use a mechanism called **self-attention** which allows them to represent each word with respect to its current context (cf. Vajjala et al. 2020, pp. 25–26).

Through self-attention, these models are able to use information from (theoretically) arbitrarily large contexts, without having to pass it sequentially through recurrent connections as in an RNN (cf. Jurafsky and Martin 2021g, p. 17).

When processing an input item, the self-attention layer has access to the item itself and all of the inputs before it, but not to the succeeding inputs.[1] The computation for each item is carried out independently, which allows

[1]This only applies to unidirectional, backward looking self-attention layers. In bidirectional architectures such as BERT, the considered context is not limited to the preceeding inputs. See Devlin et al. 2018 for further explication.

to run a high number of computations in parallel (cf. Jurafsky and Martin 2021g, p. 18. Transformer models thus allow for parallelization which is an advantage in terms of computational speed and efficiency compared to prior approaches.

Attention in transformer models can be conceptualized as '[comparing] an item to a collection of other items in a way that reveals their relevance in the current context.' (ibid., p. 18). Self-attention then means to compare a given item to other elements within a sequence (cf. ibid., p. 18).

In transformer models, each input embedding is considered in three different roles (cf. ibid., p. 19:

- As a **query**, the input embedding represents the current focus of attention and is compared to all of its preceding inputs.
- If the input embedding is among the preceding inputs, it is compared to the current query (focus of attention). In this case, it represents a **key**.
- Whenever we use the embedding for computing the current output, it is referred to as **value**.

Figure 2.5 illustrates the process of calculating the value for the third element in a sequence including the key, query, and value vectors for the current item x_3 and the preceding items x_1 and x_2. We can see that after generating these vectors, they key-query comparisons are carried out, which are then scaled using the softmax function. The resulting values are then multiplied by the value vector, and summed up to obtain the output vector, representing the output vector y_3.

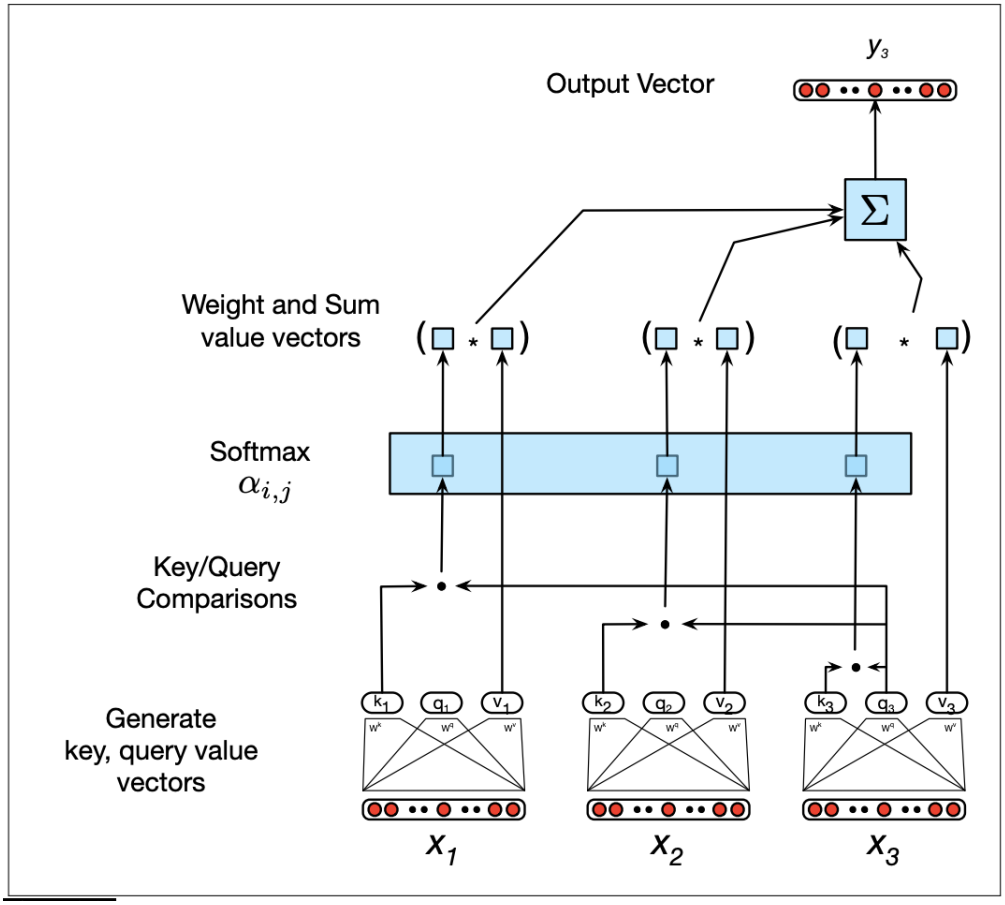

Figure 2.5: Calculating the value y_3 for the third element x_3 in a sequence (using left-to-right-attention); taken from Jurafsky and Martin 2021g, p. 20

They way context is conceptualized and modelled in this architecture is thus fundamentally different than in sequential models. By extending the concept of self-attention to multiple heads, transformer models are furthermore able to consider different aspects of relationships between words at the same time. This mechanism is called multi-head attention, and implemented in the form of multiple, parallel self-attention layers which are located at the same depth of the network. Each of these heads learns its own parameters, allowing them to pay attention to different ways in which the words in the

input relate to each other (cf. Jurafsky and Martin 2021g, p. 23).

Transformer models are composed of so-called transformer blocks. All of them have the same input und output dimensions, which allows to stack them on top of each other (cf. ibid., p. 22).

These transformer blocks are themselves multilayer networks, containing the self-attention layers, simple linear layers, and feedforward networks. As a mechanism to improve training performance, residual connections and layer normalization are implemented in each block, allowing to pass information directly to the next layer, and keeping the output values of each hidden layer in a certain range (cf. ibid., p. 22).

The great advantages of transformer models compared to sequential models are thus that they allow for parallelization and that the model context in a more complex, multi-faceted way. However, the way attention is computed makes the process expensive for long inputs, since for every input, each pair of tokens needs to be compared. This is why most models are in practice limited to a certain input length (cf. ibid., p. 21).

Furthermore, since transformer models do not work sequentially, the information about the positional context of each item in the input is lost. To provide this information, transformer models use additional positional embeddings to include this information. These embeddings are learned during training, just like the word embeddings, and encode information about the context for each input item (cf. ibid., pp. 23–25).

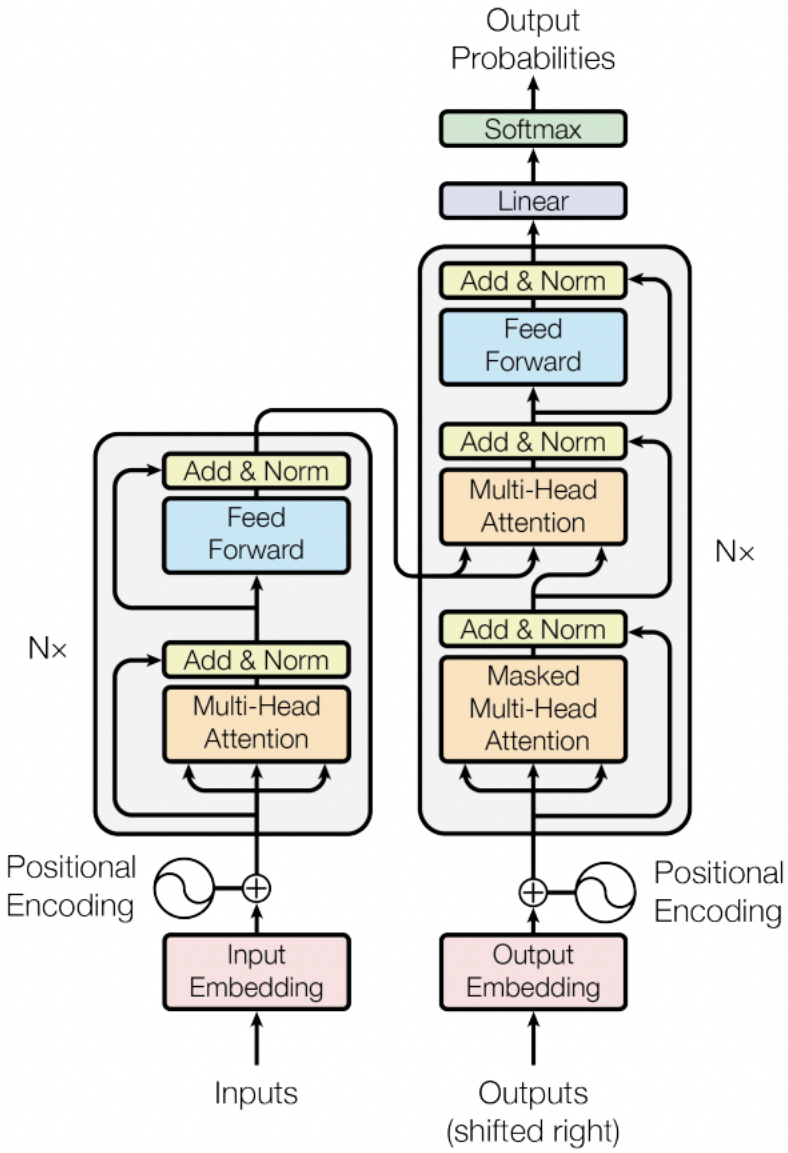

Figure 2.6: Transformer model architecure as introduced by Vaswani et al. 2017

Transformers are usually trained to serve as large language models, on different tasks such as next sentence prediction (NSP) or masked language modeling (MLM). The can also be used for sequence labeling tasks such as

NER. This is achieved by first training a language model on large amounts of unlabelled text data, which is called pre-training. Pre-training allows the model to incorporate rich information about language in general, which allows it to easier learn specific tasks, called downstream tasks, on smaller datasets later. This second step is called fine-tuning ((cf. Jurafsky and Martin 2021g, pp. 27–28). Fine-tuning requires labeled data, but can be achieved with datasets of much smaller size, since the model already learned a lot about a given language during pre-training.

2.5 Summarization

Chapter 3

Qualitative Exploration of Existing Solutions

In order to explore the potentials and limitations of existing German-language models for Named Entity Recognition, two models were selected for a basic qualitative exploration of their results with some sample texts.

While a quantitative evaluation would be the ideal approach, this is not possible since there are no existing (German-language) models which support all the entity types relevant for the NER task as defined here. Precisely, most of the models are limited to the four common entity types PERSON, LOCATION, ORGANIZATION, and OTHER/MISC. To the best of my knowledge, there are no existing models supporting the detection of the entity types TIME, DATE, and ACTION. An exception is the English-language spaCy model which supports TIME and DATE recognition. Against this background, iI will explore the potentials of machine translation to allow for the use of well-resourced English NER models for the recognition of named entities in the selected example texts.

3.1 German-language Named Entity Recognition with spaCy and Flair

The models explored in this chapter were selected from the library spaCy and the framework FLAIR. The models were chosen due to their easy applicability, allowing for fast exploration and analysis, and because both of them support German- and English-language named entity recognition.

3.1.1 The German-language spaCy model

spaCy is a free open-source library for Natural Language Processing developed for use in production. The library is extensively documented and thus serves as an easy starting point for beginner, while also claiming to be designed specifically for use in production. The library supports more than 66 languages (including German), comes with pre-trained word vectors, and supports a variety of NLP tasks such as part-of-speech tagging, text classification, and named entity recognition (cf. ExplosionAI 2022c). The German spaCy model supports to the common entity types are `LOC`, `MISC`, `ORG`, and `PER` (cf. ExplosionAI 2022d).

The central component of the model's NER pipeline is spaCy's Entity Recognizer which 'identifies non-overlapping labelled spans of tokens' (ExplosionAI 2022b) and uses a transition-based algorithm which assumes that the most relevant information about an entity is close to its first token (cf. ibid.). The underlying model is a neural network state prediction model, spaCy's TransitionBasedParser. Transition-based parsing can be described as 'an approach to structured prediction where the task of predicting the structure is mapped to a series of state transitions.' (ExplosionAI 2022e). The model is composed of two to three subnetworks which take care of token-vectorization, feature-specific vectorization for each token-feature pair and then computing the state representation from them, and (optionally) a subnetwork which predicts scores from the state representation (otherwise the outputs from the

second subnetwork will be used for prediction) (cf. ExplosionAI 2022e).

The model was trained on the WIKINER corpus, a free, multilingual corpus created from Wikipedia articles which were automatically annotated and thus are silver standard training data (cf. Nothman et al. 2013). According to the official documentation, the model achieved an F1 score of 0.84 (cf. ExplosionAI 2022d).

3.1.2 The German-language Standard NER Flair Model

The FLAIR framework is an NLP library released in 2018, developed to 'facilitate training and distribution of state-of-the-art sequence labeling, text classification and language models' (Akbik et al. 2019). The framework's aim is to provide an interface to easily use different types of word and document embeddings (cf. ibid.).

The German-language FLAIR model for NER uses a combination of GloVe embeddings (cf. Pennington, Socher, and Manning 2014 and Flair embeddings to create the word embeddings. The conceptual approach behind the Flair embeddings is described in detail in Akbik, Blythe, and Vollgraf 2018. It can briefly be characterized as an approach which creates 'contextual string embeddings' (ibid.) which take into account the context of a given text, resulting in different embeddings for the same word depending on its context in a document (cf. ibid.). The word embeddings passed to a BiLSTM-CRF sequence labeling model which takes care of the final downstream sequence labeling task (cf. ibid.).

The German Flair model used here is the German standard 4-class NER model by Flair. The model was trained on the ConLL 2003 corpus, and reached an F1 score of 87.94 according to the model's HuggingFace documentation (cf. Akbik, Blythe, and Vollgraf 2022b).

3.1.3 Example Data

For the exploration of the two models, ten example messages were randomly selected from the Telegram channel 'Demotermine' (*Demo dates*).

The channel was selected because a prior qualitative exploration of sample messages indicated that its content consists mainly of information about and mobilization for demonstrations and others forms of protests against the German COVID-19 containment measures and could thus be expected to contain a lot of relevant entities.

3.1.4 Implementation

For exploring the German medium-size spaCy model, a simple code was implemented to predict and print the detected entities:

```
import spacy
nlp = spacy.load("de_core_news_md")

def get_entities(texts):
    for text in texts:
        doc = nlp(text)
        if doc.ents:
            print("Text: ", text, '\n')
            print("The following NER tags are found: ",
            [(ent.text, ent.label_) for ent in doc.ents],
            '\n\n')
get_entities(texts)
```

The next code section shows the implementation for the Flair model:

```
from flair.models import SequenceTagger
from flair.data import Sentence

tagger = SequenceTagger.load("flair/ner-german")

def get_entities(texts):
    for text in texts:
        # transform example text to Sentence
        sentence = Sentence(text)
        # use tagger to predict NER tags in sentence
        tagger.predict(sentence)
        print(sentence, '\n')
        print('The following NER tags are found:')
        for entity in sentence.get_spans('ner'):
            print(entity)
        print('\n\n')
get_entities(texts)
```

3.1.5 Results

Table 5.1 shows the results of both models for the example texts. I would like to emphasize that due to the small size of the explored sample, these explorations are rather anecdotal and cannot be generalized without further quantitative evaluation on a larger sample. However, the following observations can be made when looking at the results:

The spaCy model produces more false positives overall: It falsely tags the sequences 'Endkundgebung' (final rally), 'GEHT DEMONSTRIEREN' (go demonstrate), 'Dresdner &', 'Videos', 'Berliner', 'Versammlungsfre' (freedom of assembly, truncated), 'Einfach' (simple/simply), 'Der Mann mit dem Schild' (the man with the sign), and 'Einsatzwägen' (patrol cars) as entities.

The Flair model only falsely tags 'FU' and 'Demo-Berlin'. The sequence 'FU' was originally part of the German word 'FÜR' and apparently was

modified by the model's tokenizer which is applied when creating a Sentence object from the input text. The sequence 'Demo-Berlin' contains a location, which makes the model's mistake to some extent plausible.

It can also be observed that the spaCy model sometimes assigns the wrong tag to a correctly detected entity. This applies for the sequences 'Blankenfelde Mahlow' which is not a person as suggested by the model, but a location.

Regarding false negatives, it can be observed that the sapCy model does not detect 'Freiburg' in the first message. On the other hand, it recognizes parts of the sequence 'Platz der alten Synagoge' (square of the old synagogue) as a location. The Flair model correctly detects Freiburg, but misses the mentioned square.

Even though the sample is too small for generalization, this first exploration casts strong doubts on the claim that spaCy offers production-ready components for NER.

It can be hypothesized that both the different model architectures and the different corpora used for training result in the better performance of the Flair model compared to the spaCy model.

3.2 Exploring the Potentials of Machine Translation for NER

Even though the spaCy model explored in the previous section provided rather dissatisfying results, its English equivalent has one advantage, since it supports more entity types, including DATE and TIME, meaning that it might be useful for extracting dates and times, as they are relevant for the aim of this thesis. Also, since more training data is available for English-language NLP models, exploring the potentials of machine translation for NER seems worth a try.

Table 3.1: Detected entities for German-language example messages

No.	Text	spaCy/de_core_news_md	flair/ner-german
1	Freiburg Aufzug mit Endkundgebung am Platz der alten Synagoge . Live Musik ! Details 19062021 Raus auf die Straße	Endkundgebung[LOC], alten Synagoge[LOC]	Freiburg[LOC]
2	GEHT DEMONSTRIEREN ! - Message von Elijah Tee an die Dresdner & anderen Demonstranten in Deutschland Morgen am 13.06. um 16.30 Uhr und bundesweit überall , wo es Mitdenker gibt . - Bilder und Videos hochladen - Diskussion Fahren oder Mitfahren	GEHT DEMONSTRIEREN [ORG], Elijah Tee[PER], Dresdner &[LOC], Deutschland[LOC], Videos[MISC]	Elijah Tee[PER], Deutschland[LOC]
3	15831 Blankenfelde Mahlow Berliner Speckgürtel Ausfahrt Lichtenrade am 14.03.2021 14032021 Raus auf die Straße	Blankenfelde Mahlow[PER], Berliner[MISC]	Blankenfelde Mahlow[LOC], Lichtenrade[LOC]
4	2021 10 11 Sven Liebich fragt Ministerpräsident Haseloff nach dessen Verständnis von Versammlungsfre - Sven Liebich 0:23 Raus auf die Straßen Übersicht / Overview	Sven Liebich[PER], Haseloff[PER], Versammlungsfre[PER], Sven Liebich[PER]]	Sven Liebich[PER, Haseloff[PER], Sven Liebich[PER]
5	FÜR ALLE DEMOKRATEN GEFAHR : Die Polizei blockiert den Stern ! Deswegen laufen die Menschen vom Ernst-Reuther-Platz nach Südosten zum Breitscheitplatz und dort weiter auf den Kudamm ! ! Schließt euch an : Neue Kantstraße direkt Richtung Osten auf den Breitscheitplatz ! Von der Straße des 17. Juni Richtung Süden zum Kudamm !	DEMOKRATEN[ORG], Ernst-Reuther-Platz[LOC], Breitscheitplatz[LOC], Kudamm ! ![LOC]), Neue Kantstraße[LOC], Breitscheitplatz[LOC], Kudamm ![LOC]	FU[ORG], Ernst-Reuther-Platz[LOC], Breitscheitplatz[LOC], Kudamm[LOC], Kantstraße[LOC], Breitscheitplatz[LOC], Kudamm[LOC]
6	Dr. Gunter Frank : Wie Moralismus dazu führt das man Feinde sieht und vernichten will — Antihygienista - Offene Gesellschaft Kurpfalz 15:04 Raus auf die Straßen Übersicht / Overview	Gunter Frank[PER], Antihygienista[LOC], Offene Gesellschaft Kurpfalz[ORG]	Gunter Frank[PER], Antihygienista[ORG], Offene Gesellschaft Kurpfalz[ORG]
7	Einfach nur Krank solche Politiker ! Was sollen wir mit ihm machen wenn unsere Kinder anhand dieser Impfung nen schaden erleiden ? Wer ist dann dafür verantwortlich , wen schmeißen wir dann in den Knast und nehmen sein ganzes Geld weg , sollen wir das mit dir machen ? Raus auf die Straße	Einfach [MISC]	*no entities detected*
8	27.11.2021 Frankfurt Durchsagen von gemischt mit Musik . Super Idee Für mehr Berichte folgen auf ... Raus auf die Straßen Übersicht / Overview	Frankfurt[LOC]	Frankfurt[LOC]
9	Demo-Berlin : Der Mann mit dem Schild wurde soeben von der Polizei festgenommen . Circa 7 Einsatzwägen sind vor Ort und umzäunen den Park großräumig . Am Humboldthain gibt es nichts mehr zu gewinnen . # b2908 Raus auf die Straßen	Der Mann mit dem Schild[LOC], Einsatzwägen[LOC], Humboldthain[LOC]	Demo-Berlin[LOC], Humboldthain[LOC]
10	Anthony Fauci : Definition von “ vollständig geimpft ” könnte geändert werden — Epoch Times Nachrichten 6:59 Raus auf die Straßen Übersicht / Overview	Anthony Fauci[PER], Times Nachrichten[MISC]	Anthony Fauci[PER], Epoch Times Nachrichten[ORG]

3.2.1 The English-language spaCy Model

The English-language spacy model supports 18 entity types: (`CARDINAL`, `DATE`, `EVENT`, `FAC`, `GPE`, `LANGUAGE`, `LAW`, `LOC`, `MONEY`, `NORP`, `ORDINAL`, `ORG`, `PERCENT`, `PERSON`, `PRODUCT`, `QUANTITY`, `TIME`, and `WORK_OF_ART`).

Table 3.2 shows the definition for each entity type (obtained for each entity via the command `spacy.explain{'<ENTITY>'}`:

Table 3.2: Overview of spaCy's entity types

Entity type	**Definition**
CARDINAL	Numerals that do not fall under another type.
DATE	Absolute or relative dates or periods.
EVENT	Named hurricanes, battles, wars, sports events, etc.
FAC	Buildings, airports, highways, bridges, etc.
GPE	Countries, cities, states.
LANGUAGE	Any named language.
LAW	Named documents made into laws.
LOC	Non-GPE locations, mountain ranges, bodies of water.
MONEY	Monetary values, including unit.
NORP	Nationalities or religious or political groups.
ORDINAL	"first", "second", etc.
ORG	Companies, agencies, institutions, etc.
PERCENT	Percentage, including "%".
PERSON	People, including fictional.
PRODUCT	Objects, vehicles, foods, etc. (not services).
QUANTITY	Measurements, as of weight or distance.
TIME	Times smaller than a day
WORK_OF_ART	Titles of books, songs, etc.

The training data sources listed in the model's official documentation suggest that the NER training data was the OntoNotes 5 corpus (cf. Ralph Weischedel 2013). The model achieved an F1 score of 0.85 for NER.

3.2.2 The English-language Standard NER Flair Model

The architecture of the standard English-language Flair model is the same as for the standard German model described in section 3.1.2. It was trained on a corrected version of the CoNLL 2003 corpus and achieved an F1 score of 93.06 (cf. Akbik, Blythe, and Vollgraf 2022a).

3.2.3 Implementation

To explore the potentials of the medium-size English spaCy model and the Flair English-language standard model, the example texts were translated into English using the DeepL API:

```
import deepl
from src.config import deepl_credentials
auth_key = deepl_credentials.auth_key

# initialize and authenticate translator
translator = deepl.Translator(auth_key)

# method to get translated texts
def translate(text, translator, target_lang, formality="less"
                                ):
    result = translator.translate_text(text, target_lang=
                                    target_lang)
    return result.text

# apply translation to cleaned texts
df['text_EN'] = df.cleaned_text.apply(lambda x: translate(x,
                                translator, 'EN-US'))
```

The translated texts were then fed to the English-language spaCy and Flair model, using the same code as described in section 3.1.4.

3.2.4 Results

Table 3.3 shows the results of both models for the translated example texts.

3.3 Summarization

potentials and limitations / observed errors - spacy - Flair explorative evaluation

Table 3.3: Detected entities for English-language example messages

No.	Text	spaCy/en_core_web_md	flair/ner-english
1	Freiburg procession with final rally at the square of the old synagogue . Live music ! Details 19062021 Out on the street	*no entities detected*	Freiburg[LOC]
2	GO DEMONSTRATE ! - Message from Elijah Tee to the Dresdeners & other demonstrators in Germany Tomorrow on 13.06. at 16.30 clock and nationwide everywhere , where there are fellow thinkers . - Upload pictures and videos - discussion drive or ride along	Germany[GPE], Tomorrow[DATE], 13.06[CARDINAL], 16.30[CARDINAL]	Elijah Tee[PER], Dresdeners[MISC], Germany[LOC]
3	15831 Blankenfelde Mahlow Berlin Speckgürtel Lichtenrade exit on 03/14/2021 14032021 Out on the road	15831[DATE], Blankenfelde Mahlow Berlin[PERSON], Speckgürtel Lichtenrade[PERSON], 03/14/2021 14032021[DATE]	"Blankenfelde Mahlow Berlin Speckgürtel Lichtenrade"[MISC]
4	2021 10 11 Sven Liebich asks Prime Minister Haseloff about his understanding of assembly free - Sven Liebich 0:23 Raus auf die Straßen Overview / Overview	Sven Liebich[ORG], Haseloff[PER], Straßen Overview / Overview[ORG]	Sven Liebich[PER], Haseloff[PER], Sven Liebich[PER], Raus auf die Straßen Overview[ORG]
5	FOR ALL DEMOCRATS DANGER : The police blocks the Stern ! That's why people run from Ernst-Reuther-Platz to the southeast to Breitscheitplatz and there further to Kudamm ! ! Join : Neue Kantstraße directly to the east to Breitscheitplatz ! From the Straße des 17. Juni southwards to the Kudamm !	DEMOCRATS[NORP], Ernst-Reuther-Platz[ORG], Breitscheitplatz[PERSON], Kudamm[FAC], Neue Kantstraße[PERSON], Straße[GPE], 17[CARDINAL], Kudamm[FAC]	Stern ![MISC], Ernst-Reuther-Platz[LOC], Breitscheitplatz[LOC], Kudamm[LOC], Neue Kantstraße[ORG], Breitscheitplatz[LOC,]Straße des[LOC], Juni[LOC], Kudamm[LOC]
6	Dr. Gunter Frank : How moralism leads to the fact that one sees enemies and wants to destroy - Antihygienista - Offene Gesellschaft Kurpfalz 15:04 Raus auf die Straßen Übersicht / Overview	Gunter Frank[PER], 15:04 Raus auf[PER], Straßen Übersicht / Overview[ORG]	Gunter Frank[PER]
7	"Just sick such politicians ! What should we do with him if our children suffer from this vaccination ? Who is responsible for it , who do we throw in jail and take away all his money , shall we do that with you ? Out on the street"	*no entities detected*	*no entities detected*
8	27.11.2021 Frankfurt announcements of mixed with music . Super idea For more reports follow on ... Out on the streets Overview	Frankfurt[GPE]	Frankfurt[LOC]
9	Demo-Berlin : The man with the sign has just been arrested by the police . About 7 emergency vehicles are on site and fence the park extensively . At the Humboldthain there is nothing more to win . # b2908 Out on the streets	Humboldthain[PER], #[CARDINAL]	Humboldthain[LOC]
10	Anthony Fauci : Definition of " fully vaccinated " could be changed - Epoch Times News 6:59 Out on the Streets Overview / Overview.	Anthony Fauci[PER], the Streets Overview / Overview[ORG]	Anthony Fauci[PER], Epoch Times News[ORG], Streets Overview[MISC]

Chapter 4

Building a Custom Model for NER in Telegram Channels

4.1 Building a Domain-specific Dataset: The Telegram Corpus

For the creation of a dataset suitable for the aim of this thesis, the Telegram channel 'Demotermine' (Demo dates) was selected.

The channel was chosen because a prior qualitative exploration in the context of the research prject 'Digitaler Hass'[1] revealed that its content consists mainly of information about and mobilization for demonstrations and others forms of protests against the German COVID-19 containment measures. The extended channel name 'Demo-Termine & Kontakte, Gleichgesinnte treffen - Demokalender, Spaziergänge, Mahnwachen, Meditation *Rücktritt Bundesregierung!' (Demo dates & contacts, meet like-minded - Demo calendar, walks, vigils, meditation *resignation federal government!) also reflects this focus. The channel is thus a promising source of text data containing a lot of entities relevant for the aim of this thesis.

[1] https://www.digitalerhass.de/

4.1.1 Exploring the Raw Data

The data collected includes messages from the start of the channel until December 19, 2021. By the beginning of June 2022, the channel had around 52.900 subscribers.

The dates of the collected messages range between March 3rd, 2018 and December 19th, 2021 (last day included in the data collection process). Even though the channel existed before the outbreak of the COVID-19 pandemic, an exploration of message frequencies clearly reveals that the channel gained relevant momentum at the beginning of the pandemic in Germany in spring 2020, as can be seen in figure **??**. The overall dataset consists of 31.430 rows of data. After the removal of text duplicates and empty rows, 27.627 rows remained for further processing.

4.1.2 Preprocessing

The overall text cleaning process was restricted to an absolute minimum in order to create a sample data which is as realistic as possible with regard to the task of recognizing named entities in social media text. This basically included the removal of newlines and splitting up of compound hashtags into separate hashtags (so that e.g. #food#delicious would be split up into two different tokens #food and #delicious).

`@user_mentions` or `#hashtags` were deliberately included, and the special characters and words of these tokens were not split up into separate tokens. In-word hyphens (such as in Demo-Termine) were also not split up from their surrounding words. URLs and emojis were included as well, and the original capitalization of words was left unchanged.

For preprocessing, the tokenizer of the German medium-size spaCy model (`de\_core\_news\_md`) was applied, using a customized token match pattern. The following code parts were applied to all rows in the dataset, storing the results in a separate `cleaned_text` column. :

```
import re
import spacy
from spacy.tokenizer import _get_regex_pattern

def preprocess_text(s, nlp):
    # remove newlines
    s = ' '.join(s.split())
    # split up compound hashtags, e.g. #food#delicious
    s = re.sub(r'#', r' #', s)
    doc = nlp(s)
    # tokenize keeping all tokens, no removal of emojis
    words = [token.text for token in doc]
    # join tokens back together
    result = " ".join(words)
    #replace duplicate whitespaces
    result = re.sub(r'  ', '', result)
    return result
```

Preventing the special characters in hashtags and user mentions from being split up was intended to serve for creating a realistic social media corpus and allow to include these special characters into the labels assigned during the annotation process. However, evaluation of training results indicate that it might indeed make sense to split up these special characters and simply treat them as separate tokens which are not treated as part of the tagged named entities.

After preprocessing, all rows which did not contain any results for cleaned text were removed, leaving 22.803 messages for sample selection.

4.1.3 Sample Selection

For sample selection, first the text length for each text was obtained to enable filtering texts based on their length. The second step consisted of detecting the language for each text in the dataset in order to be able to filter out non-German texts. For this, the `Python` package `langdetect` was applied to

the cleaned texts.

Results show that German-language content is clearly predominant with $\sim$ 83% (22.803 messages), followed by English ($\sim$ 3.8%, 3.823 messages), and then French, Italian, and Dutch (less than 1% each).

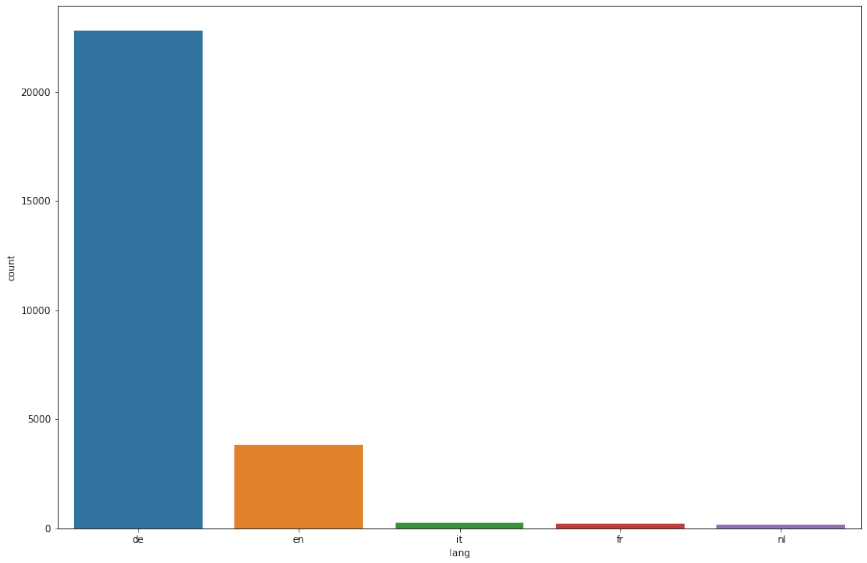

Figure 4.1: Five most frequent languages

It should be noted that the language detection was sometimes flawed due to the fact that the channel contains a number of mixed-language texts resulting in messages being classified as German while actually also consisting of a relevant fraction of e.g. English or other non-German languages. These mixed-language messages were filtered out later during the annotation process. However it could also be argued that they should be included in the final training data, since they represent realistic social media date and NER systems should be able to handle such kinds of texts as well.

As a preparatory step for the final sample selection, the dataset was

reduced to only German-language texts (22.803 messages remaining). Out of these messages, only those with a minimum length of 100 characters (in the `cleaned_text` column) were selected (14.605 messages remaining).

An exploration of the text length of all German-language messages shows that the messages have a median text length of 207 characters, and 75% of the messages have a text length of up to 400 characters. The minimum text length is 5 characters, the longest text consists of 4458 characters.

Considering these insights, only messages consisting of at least 100 characters were selected (20.105 messages remaining). Out of these, twenty percent of the messages were randomly selected (4021 messages remaining).

Insights from prior research with Telegram data in the context of the project 'Digitaler Hass' revealed that channels often contain a relatively high number of similar messages. To avoid redundancy in the sample, the similarity of the remaining messages was computed based on the Levenshtein distance:

```
import Levenshtein as lev
import pandas as pd
from datetime import datetime
from itertools import combinations


  def get_levenshtein_results(df, column='cleaned_text'):
    lev_df = pd.DataFrame()
    new_df = pd.DataFrame(combinations(df[column],2), columns
                                    =["text_1", "text_2"])
    new_df["lev_score"] = new_df.apply(lambda x: lev.ratio(x[
                                    0],x[1]), axis=1)
    lev_df = pd.concat([lev_df, new_df], axis=0).reset_index(
                                    drop=True)
    return lev_df


def get_message_ids(df, column='message_id'):
```

```
    lev_df_ids = pd.DataFrame()
    new_df_ids = pd.DataFrame(combinations(df[column],2),
                                    columns=["message_id_1", "
                                    message_id_2"])
    lev_df_ids = pd.concat([lev_df_ids, new_df_ids], axis=0).
                                    reset_index(drop=True)
    ts_after = datetime.now()
    return lev_df_ids


def combine_levenshtein_results_with_message_ids(lev_df,
                                lev_df_ids):
    levs = pd.DataFrame()
    levs = pd.concat([lev_df,lev_df_ids], axis=1)
    ts_after = datetime.now()
    return levs


def remove_similar_messages(lev_ids_df, sample_df, treshold):
    second_ids = lev_ids_df[lev_ids_df['lev_score'] >=
                                    treshold].message_id_2.
                                    tolist()
    ids_to_drop = list(set(second_ids))
    result_df = sample_df[~sample_df.message_id.isin(
                                    ids_to_drop)]
    return result_df
```

A treshold of 0.8 was defined. If the Levensthein score between for a pair of messages was greater than or equal to this treshold, the second message was removed from the sample.

The remaining sample after this step consisted of 3.095 messages.

In a second iteration, an additional sample of short texts was selected. This was supposed to reflect the fact that social media texts can be of very short length and thus should be included in the training data for NER systems. The sampling process for these short texts was similar to the first

sampling iteration, except for the sampling size which was 50% instead of 20% in the first iteration. After random sampling and removing similar messages based on the Levenshtein score, a sample of 540 short messages remained and was added to the overall sample dataset.

4.1.4 Annotation

Data annotation turned out to be among the most challenging subtasks. Existing annotation schemes for NER vary greatly in terms of entity types, detailedness and precision of the provided definitions, granularity, and the handling of social media-specific phenomena like user mentions, urls, or hashtags.

For this thesis, a custom annotation schema had to be developed. The schema which includes basic definitions, detailed instructions as well as several examples can be found in the appendix (see 6.2).

For carrying out the annotation, the tool Labelstudio[2] was used.

The tag set covers the six entity types person, location, organization, time, date and action. Following the IOB tagging scheme, a B- and an I-tag was provided for each entity type, resulting in a total of 12 tags. The O-tag (denoting tokens outside of an NE) was assigned automatically to all tokens who remained unlabeled at the end of the manual annotation process.

4.1.5 Telegram Corpus Statistics

The final annotated corpus consists of 500 documents with a total of 2.509 annotated named entities. Table 4.1 shows the distribution of the different entity types among the corpus after splitting up the dataset into a training (80%), test (10%), and validation set (10%). We can see that locations are by far the most often mentioned entity types (∼51%), followed by dates (∼17%), and action mentions (∼12%).

[2]https://labelstud.io/

Table 4.1: Distribution of entities in Telegram corpus

	LOC	PER	ORG	DATE	TIME	ACTION	total
train	1.140	76	132	321	188	253	2.110
test	66	17	26	62	19	29	219
validation	73	17	18	40	9	23	180
total	1.279	110	176	423	216	305	2.509

4.2 Additional Data: The DFKI Smartdata Corpus

The Telegram corpus is very small of size, compared to other corpora used for training NER models. The GermEval 2014 corpus for example consists of 31,300 sentences containing more than 41.000 named entities (cf. Benikova et al. 2014. Due to the limited resources in terms of times and personnel, manually annotating larger amounts of data for this thesis was not possible. Against this background, additional data from the Smartdata corpus by Schiersch et al. 2018 was taken and adapted to fit the task at hand.

4.2.1 Corpus Description

The Smartdata corpus consists of 2.598 German-language documents sampled from online resources covering three genres: newswire texts, Twitter, and traffic reports via RSS feeds from radio stations, police and railway companies (cf. ibid.). The dataset was created by researchers at the German Research Center for Artificial Intelligence (Deutsches Forschungszentrum für Künstliche Intelligenz, DFKI) and is available under a permissive CC-BY 4.0 license.[3] The corpus contains a total of 22.075 named entities (cf. ibid.).

Even though the corpus is still relatively small of size compared to the above-mentioned standard datasets such as GermEval2014 or CoNLL2003, it is useful for several reasons: First, the Smartdata corpus is explicitly designed for the recognition of events and therefore includes time and date

[3]https://creativecommons.org/licenses/by/4.0/

mentions, extending the commonly used entity types. Second, unlike the standard datasets, it consists of online documents, making it more similar to the Telegram corpus than corpora created from classic news articles.

However, the tag set of the corpus is not readily combinable with the Telegram corpus, since the Smartdata corpus is based on a (partially) fine-grained annotation scheme: Organizations were annotated as organization or organization-company, and locations were annotated as location, location-city, location-street, location-route, and location-stop. Furthermore, the dataset contains entity types which are not of interest for my task, such as duration, distance, disaster type, or trigger events. It should also be noted that even for the same entity types, the two corpora might differ slightly since the development of the annotation scheme and the annotation process itself were carried out separately. Another disadvantage is that the Smartdata corpus does not contain the entity type action (or any similar type). This results in an underrepresentation of this specific class when merging both datasets and might most probably cause problems during training, since there will not be enough examples for the model to learn, and mentions of actions occuring in the Smartdata corpus do not have a corresponding tag.

4.2.2 Label Mapping

For merging the two corpora, the labels of the Smartdata corpus were mapped to those of the Telegram corpus. This was done by mapping all entities not of interest (e.g. duration) to the O-tag, and mapping the fine-grained entity labels to their coarse-grained match, provided that the definition of the fine-grained entity was considered to sufficiently match the definition used in the Telegram corpus. Table 4.2 shows the mapping between the two labels schemas in detail.

Table 4.2: Mapping of label schemas

Telegram Label	**Original Smartdata Label**
B-DATE	B-DATE
I-DATE	I-DATE
B-LOC	B-LOCATION, B-LOCATION_CITY, B-LOCATION_STOP, B-LOCATION_STREET
I-LOC	I-LOCATION, I-LOCATION_CITY, I-LOCATION_STOP, I-LOCATION_STREET
B-ORG	B-ORGANIZATION, B-ORGANIZATION_COMPANY
I-ORG	I-ORGANIZATION, I-ORGANIZATION_COMPANY
B-PER	B-PERSON
I-PER	I-PERSON
B-TIME	B-TIME
I-TIME	I-TIME
O	B-DISASTER_TYPE, I-DISASTER_TYPE, B-DISTANCE, I-DISTANCE, B-DURATION, I-DURATION, B-LOCATION_ROUTE, I-LOCATION_ROUTE, B-NUMBER, I-NUMBER, B-ORG_POSITION, I-ORG_POSITION, B-TRIGGER, I-TRIGGER

4.2.3 Adjusted Smartdata Corpus Statistics

After carrying out the label mapping, the adjusted Smartdata corpus contains a total of 14.557 entities, distributed as shown in table 4.3. We can see that as in the Telegram corpus, location mentions represent the largest class (∼47%). The next largest proportions are organizations (∼27%), and dates (∼13%).

Table 4.3: Distribution of entities in Smartdata corpus after label mapping

	LOC	**PER**	**ORG**	**DATE**	**TIME**	**ACTION**	**total**
train	5.526	978	3.223	1.511	556	0	11.794
test	681	100	307	168	69	0	1.325
validation	688	127	387	167	69	0	1.438
total	6.895	1.205	3.917	1.846	694	0	14.557

4.3 The Final Corpus

In order to use both datasets for training a custom transformer model, some preparatory steps were carried out, including label mapping and creating Hugging Face Transformers datasets from both corpora.[4] Combining the Telegram corpus and the adjusted Smartdata corpus results in a total of 3.098 documents including 17.066 named entities with a class distribution of as shown in table 4.4.

Resulting from the stark proportion of location mentions in both datasets, here once again entities of type location make up for almost half of all entities (∼48%). The next most frequent entity types are organizations (∼24%), and dates (∼13%). Less frequently mentioned are persons (∼8%), and times (∼5%). Entities of type action make up for less than 2%, which is not surprising given the fact that this tag only exists in the far smaller Telegram corpus.

Table 4.4: Distribution of entities in final corpus

	LOC	**PER**	**ORG**	**DATE**	**TIME**	**ACTION**	**total**
train	6.666	1.054	3.355	1.832	744	253	13.904
test	747	117	333	230	88	29	1.544
validation	761	144	405	207	78	23	1.618
total	8.174	1.315	4.093	2.269	910	305	17.066

4.4 Overview of Used Models

There is a large number of different architectures for transformer models, resulting in an even larger number of available models.[5]

[4]For implementation details see: `https://github.com/eli-quereli/ner-german-telegram/tree/main/src/notebooks/4.2_training/01_prepare_datasets`

[5]By the time of writing this thesis, the Hugging Face Hub lists more than 62.000 of pretrained models available for download: `https://huggingface.co/models`, accessed on 2022/08/06.

The three main architectures for transformer models are encoders, decoders, and encoder-decoders. Encoder models are still the most commonly used, both in research and for business applications (cf. Tunstall et al. 2022, p. 79), and the models selected for the experiments here as well rely on the encoder architecture. These general architecture types can be further differentiated into different model architectures such as e.g. DistilBERT or XLM models.[6]

Models which have been pretrained on different corpora and can be used e.g. for fine-tuning tasks are called checkpoints. The term 'model' is thus a general term which can refer to either a a specific architectural model skeleton or a checkpoint of a pretrained model.

The experiments in this thesis were carried out with five different pretrained models (checkpoints) from the three different encoder model architectures BERT, DistilBERT, and XLM-R. The models differ not only in terms of their underlying architecture, but also in terms of the size, sources, and languages included in the corpora used for pre-training.

4.4.1 BERT models

Bidirectional Encoder Representations from Transformers were developed by Google. The pretraining tasks for these models are masked language modeling and next sentence prediction (cf. Devlin et al. 2018). I selected two pretrained German-language BERT models for my experiments:

dbmdz/bert-base-german-uncased

Model card: `https://huggingface.co/dbmdz/bert-base-german-uncased`

This model is provided by the MDZ Digitial Library team (dbmdz) at the Bavarian State Library. It is a German-language model which was pretrained

[6] See Tunstall et al. 2022, p. 79 for an overview and ibid., pp. 57–86 for details on the different architectures.

on 16 GB of data. Data for pre-training was taken from e.g. Wikipedia, but also large online corpora created via web crawlers (cf. model card).

I decided to use the uncased version of the model (meaning that it does not pay attention to text casing), since capitalization in user-generated text data is often unreliable.

The cased version of this model was evaluated for NER fine-tuning on the CoNLL2003 corpus and the GermEval2014 corpus, achieving F1 scores of 84.52 and 86.89 on the respective test sets.[7].

deepset/gbert-base

Model card: `https://huggingface.co/deepset/gbert-base`

This German-language BERT model was trained collaboratively by a team from deepset and the above-mentioned dbmdz team. The model was pre-trained on 163.4 GB of data mainly consisting of scraped online resources (cf. Chan, Schweter, and Möller 2020). The model was evaluated for NER on the GermEval2014 dataset, achieving an F1 score of 87.98 (cf. ibid.).

4.4.2 DistilBERT model

DistilBERT models are distilled version of BERT models, designed to allow for faster training requiring less memory, yet achieving only almost the same accuracy as regular BERT models. This is done via a process called knowledge distillation (cf. Tunstall et al. 2022, p. 80).[8]

distilbert-base-multilingual-cased

Model card: `https://huggingface.co/distilbert-base-multilingual-cased`

[7] See `https://github.com/stefan-it/fine-tuned-berts-seq`, accessed on 2022/06/08

[8] Cf. Tunstall et al. 2022, pp. 209–248 for details on knowledge distillation.

This model is a distilled version of a multilingual BERT model. It was pretrained on a masked language modeling task using a concatenation of Wikipedia in 104 languages and is reported to be twice as fast as its BERT counterpart.[9] The model is cased and thus considers capitalization. Unfortunately, there are no evaluation results for NER fine-tuning.

4.4.3 XLM-Roberta models

RoBERTa models modify the pretraining scheme of BERT models by dropping the task of next sentence prediction and training longer, on larger batches, and with more training data, improving the results of the original BERT models (cf. Tunstall et al. 2022, p. 80). XLMs on the other hand are cross-lingual language models. Their pretraining extends the masked language modeling task to translation language modeling by including inputs from multiple languages (cf. ibid., p. 80).

XLM-Roberta models combine these two approaches by performing multilingual pretraining with large amounts of training data (cf. ibid., p. 80).

xlm-roberta-base

Model card: `https://huggingface.co/xlm-roberta-base`

This model was pretrained on 2.5 TB of filtered data over 100 languages from a corpus created via web crawlers (cf. Conneau et al. 2019. The model was evaluated on the CoNLL2002 and ConLL2003 datasets for NER fine-tuning. The base model used here achieved an F1-score of 84.60 for German when fine-tuned on each language set separately (used to evaluate per-language performance, cf. ibid.).

[9] `https://huggingface.co/distilbert-base-multilingual-cased`, accessed on 2022/06/08.

cardiffnlp/twitter-xlm-roberta-base

Model card: `https://huggingface.co/cardiffnlp/twitter-xlm-roberta-base`

The last model is an XLM-RoBERTa model which was trained on ∼198M tweets in over thirty languages. (cf. Barbieri, Anke, and Camacho-Collados 2021). The evaluation of the model in the reference paper focuses on Twitter-specific tasks like e.g. emoji prediction, hate speech and offensive language detection, irony detection, or sentiment analysis. Unfortunately, no evaluation of NER fine-tuning is provided (cf. ibid.).

4.5 Defining a Baseline

Choosing a suitable baseline for my task is difficult due to a various reasons: Even though some of the chosen models provide evaluation results for German-language NER which could serve as a baseline, the fine-tuning set-up is not directly comparable to my one. Given the fact that the standard datasets CoNLL2003 and GermEval2014 were used in the above-mentioned fine-tuning tasks, the conditions are not comparable to my one regarding the size of the corpus, thw text genre, and the number of tags. Furthermore, the evaluation results differ for the different model architectures used, making it questionable whether it is suitable to evaluate e.g. a BERT-based model against a the F1-score of an XLM-R-based model.

The shared tasks on NER in social media texts described in 2.3.9 on the other hand are roughly comparable in terms of corpus size, genre, and the number of tags. However, they are not directly comparable in terms of technology since none of the particpating systems made use of the potentials of transformer models. Against this background, I will compare my results against two baselines. Baseline 1 takes NER fine-tuning of transformer models as a reference, using the best provided results from the chosen models, namely the `deepset/gbert-base` model. Baseline 2 refers to NER in so-

cial media texts, taking the best participating system, coined `ousia` in the W-NUT 2015 shared task as a reference (cf. Baldwin et al. 2015, Yamada, Takeda, and Takefuji 2015).

- Baseline 1: F1 = 87.98 (deepset/gbert-base)
- Baseline 2: F1 = 56.41 (ousia)

4.6 Experimental setup

For carrying out my experiments, as a first step I decided to train the chosen models on each dataset separately to obtain first insights into the models' performances. In a second step, I trained each model on the final corpus, a combined version of the Telegram and the adjusted Smartdata corpus as described in section 4.3. The next sections will describe the required steps for training and evaluation. After that, I will present the results from my experiments.

4.7 Preprocessing: Tokenization and Encoding

Like other neural networks (or any other machine learning models in general), transformer models cannot process raw strings, but require tokenized and encoded (i.e. vectorized) representations of textual data as input (cf. Tunstall et al. 2022, p. 29). This is done in the first part of the pipeline via a tokenizer. Tokenization basically refers to the act of breaking down a string into single units. There are different tokenization strategies ranging from character-based over subword-based to word-based tokenization (cf. ibid., p. 29).

Transformer models rely on subword tokenization as an attempt to combine the advantages of both character- und word-based tokenization. This

means that rare words are split up into smaller units while more frequent words are not split up. This allows to handle complex words and misspellings, but at the same time keep the input length at a processable size (cf. Tunstall et al. 2022, p. 33). Which words should be split up and which not is usually learned during the pre-training of transformer models, using different algorithms such as WordPiece, Byte-Pair Encoding (BPE), or SentencePiece (cf. HuggingFace 2022d) . Against this background, the tokenizer used for later fine-tuning needs to match the pre-training process.

The tokenizer takes care of several pre-processing steps including normalization, pretokenization, subword tokenization, and post-processing (cf. Tunstall et al. 2022, pp. 94–95).

To load a tokenizer which fits the pre-trained model, the `AutoTokenizer` class can be used by passing the model name to the `from_pretrained()` method:

```
from transformers import AutoTokenizer

model_name = 'deepset/gbert-base'
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

While the tokenizer splits up the input strings into subwords, the NE tags in the training corpus are assigned to entire tokens. We thus need to handle this by properly aligning the NE tags with their corresponding subtokens. Since every word in the input is assigned a unique `word_id` which is also assigned to the resulting subtokens of the word, we can make use of this id for tag alignment. If a subtoken has the same `word_id` as its predecessor, we can assign the value -100 as its `label_id`. This tells the cross entropy loss function to ignore these subtokens and only consider the label assigned to the first subtoken of a word (cf. ibid., p. 104).

The following method based on ibid., p. 104 was used for tokenization and label alignment of the Telegram and Smartdata corpus.

```
def tokenize_and_align_labels(examples):
```

```
    tokenized_inputs = tokenizer(
        examples['tokens'],
        padding='max_length',
        truncation=True,
        is_split_into_words=True
    )

    labels = []

    for idx, label in enumerate(examples['ner_tags']):
        word_ids = tokenized_inputs.word_ids(batch_index=idx)
        previous_word_idx = None
        label_ids = []
        for word_idx in word_ids:
            if word_idx is None or word_idx ==
                                            previous_word_idx:
                label_ids.append(-100)
            else:
                label_ids.append(label[word_idx])
            previous_word_idx = word_idx
        labels.append(label_ids)
    tokenized_inputs['labels'] = labels
    return tokenized_inputs
```

We can see above that besides the input tokens, some additional arguments are passed to the tokenizer. These arguments tell the tokenizer that the input texts is already split into separate tokens via `is_split_into_words=True`. Furthermore, since transformer models operate with fixed-size vector representations of inputs (called tensors), the tokenizer needs to be told how to handle inputs of different length. Truncation deals with long inputs by truncating them to the maximum length accepted by a given model (e.g. 512 tokens for BERT models). Padding on the other hand deals with shorter input sequences by adding a special padding token until the defined length (here: maximum length accepted by the model) is reached (cf. HuggingFace

2022c).

Before we can start to train the model, the whole dataset needs to be encoded. This can be achieved via the `map()` method of the Hugging Face Datasets library. In this step, unnecessary columns are also removed. We encode both the training and the validation split since we need the validation set in its encoded form for evaluating the training results:

```
def encode_dataset(corpus, columns_to_remove=['ner_tags', '
                                tokens', 'ner_tags_str']):
    return corpus.map(tokenize_and_align_labels,
                      batched=True,
                      remove_columns=columns_to_remove)
```

We can now load the prepared Telegram corpus and the adjusted Smartdata corpus from disk.

For fine-tuning with separate datasets, we can simply use the `encode_dataset()` method. For fine-tuning with the combined datasets, we first need to concatenate the respective training and validation splits, and then encode them. Note that the training split is being shuffled before training to randomly rearrange its values, thus ensuring that there is no unintended ordering in the data which might affect the model training).

```
train_ds = concatenate_datasets([smartdata['train'], telegram
                                ['train']])
train_ds = train_ds.shuffle(seed=42)

eval_ds = concatenate_datasets([smartdata['validation'],
                                telegram['validation']])
```

Finally, we can tokenize and encode both splits:

```
train_encoded = encode_dataset(train_ds)
eval_encoded = encode_dataset(eval_ds)
```

After encoding, the dataset includes numerical `input_ids`, representing the input tokens, and numerical `labels`, representing the correspond-

ing NE tags, as shown in figure 4.2. The tokenizer has also returned an `attention_mask` for each input document, telling which token should be attended and which not (cf. HuggingFace 2022a). Furthermore, the tokenizer created `token_type_ids`. These are not relevant for the task at hand, but used for tasks in which an input contains two sequences, e.g. for question answering or for tasks involving pairs of sentences. The `token_type_ids` tell the model to which of the two sequences a token belongs (cf. HuggingFace 2022b).

	input_ids	token_type_ids	attention_mask	labels
0	[102, 20739, 8497, 30914, 10412, 30925, 22171,...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[-100, 0, -100, -100, -100, -100, -100, -100, ...
1	[102, 17329, 14705, 18315, 17329, 19281, 30904...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[-100, 0, -100, -100, 5, -100, -100, -100, -10...
2	[102, 718, 1412, 6556, 17077, 3859, 136, 8139,...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[-100, 0, 3, -100, -100, -100, 0, 3, -100, 0, ...
3	[102, 493, 566, 1483, 1073, 818, 3035, 853, 32...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[-100, 1, 2, 2, 2, 0, 9, -100, -100, 10, 5, 0,...
4	[102, 15591, 17329, 7938, 2032, 305, 1055, 853...	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...	[-100, 0, 0, -100, -100, -100, -100, 0, 0, 0, ...

Figure 4.2: Training data after tokenization and encoding

We now have encoded our textual input, consisting of tokens and labels, into a numerical input which can be processed by the model.

4.8 Loading and Configuring a Model

For this, we need to load the model and adjust its configuration to make it fit our dataset. We do this by loading the `AutoConfig` of the pretrained model and handing over the number of labels (`num_labels`)in our dataset as well as the mapping between numerical and textual labels (`id2label`) and vice versa (`label2id`).

```
from transformers import AutoConfig
model_config = AutoConfig.from_pretrained(model_name,
                                          num_labels=tags.num_classes,
```

```
                                    id2label=index2tag, label2id=
                                    tag2index)
```

When loading a model, we need to choose a model which fits our task. Since we want to classify single tokens for NER, we choose a model for token classification and pass the adapted model configuration:

```
from transformers import AutoModelForTokenClassification
model = AutoModelForTokenClassification.from_pretrained(
                                    model_name, config=
                                    model_config)
```

4.9 Defining Training Arguments

Before we can actually start training, we will define some training arguments. We will train the model for 10 epochs, which means that the model will work through the whole dataset for ten times. We use batch size here to define the number of examples processed per device (e.g. a GPU) in each training step. A batch size of 2 is admittedly small, but was chosen to avoid running into memory errors during training. Weight decay is used as a regularization technique to prevent overfitting (see e.g. Koppert-Anisimova 2022 for a brief explanation).

We also define a callback here to implement early stopping during training. Callbacks can be used to customize the training behaviour of transformer models and take decisions during training. Here, the decision is to stop if the provided `metric_for_best_model` deteriorates the third time in a row. Training will then be stopped, and the last best model, defined via the loss on the validation set (`metric_for_best_model`) will be loaded. The remaining parameters define logging and evaluation details. To obtain comparable results, all experiments were trained with the same training arguments.

```
from transformers import EarlyStoppingCallback
```

```
NUM_EPOCHS = 10
BATCH_SIZE = 2
LOGGING_STRATEGY='epoch'
OUTPUT_DIR = f'../../../../../models/{model_name}-{
                                dataset_name}-{NUM_EPOCHS}
                                _epochs' # only for first
                                experiments
OVERWRITE_OUTPUT_DIR = True
LOG_LEVEL = 'error'
EVALUATION_STRATEGY = 'epoch'
SAVE_STRATEGY ='epoch'
WEIGHT_DECAY = 0.01 # The weight decay to apply (if not zero)
                                 to all layers except all bias
                                 and LayerNorm weights in
                                AdamW optimizer.
CALLBACK = EarlyStoppingCallback(early_stopping_patience=3)
LOAD_BEST_MODEL_AT_END = True,
METRIC_FOR_BEST_MODEL='eval_loss'
```

We use these arguments to create a `TrainingArguments` object in the next step:

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    overwrite_output_dir=OVERWRITE_OUTPUT_DIR,
    log_level=LOG_LEVEL,
    num_train_epochs=NUM_EPOCHS,
    per_device_train_batch_size=BATCH_SIZE,
    per_device_eval_batch_size=BATCH_SIZE,
    evaluation_strategy=EVALUATION_STRATEGY,
    save_strategy=SAVE_STRATEGY,
    weight_decay=WEIGHT_DECAY,
    logging_strategy=LOGGING_STRATEGY,
    disable_tqdm=False,
    load_best_model_at_end=LOAD_BEST_MODEL_AT_END,
```

```
    metric_for_best_model=METRIC_FOR_BEST_MODEL)
```

4.10 Loading a Data Collator

Next we load a data collator which takes care of padding the labels and the inputs. This step is provided here since it was part of my implementation. It should be noted however that padding already was carried out during the tokenization process which is why using a data collator is not necessary here, but should rather be understood as an alternative to padding via the tokenizer.

```
data_collator = DataCollatorForTokenClassification(tokenizer)
```

4.11 Defining Evaluation Metrics

We also need to define a method for evaluating the model's predictions on the provided validation set during training. This method, `compute_metrics`, returns the F1 score as well as precision, recall, and accuracy per epoch. It relies on a method which takes care of aligning the predictions of the model for the subtokens by ignoring the subtokens with the `label_id` -100 - analogue to the process of label alignment in section 4.7.

```
from seqeval.metrics import f1_score, precision_score,
                            recall_score, accuracy_score

def align_predictions(predictions, label_ids):
    preds = np.argmax(predictions, axis=2)
    batch_size, seq_len = preds.shape
    labels_list, preds_list = [], []

    for batch_idx in range(batch_size):
        example_labels, example_preds = [], []

        for seq_idx in range(seq_len):
            # Ignore label IDs = -100
            if label_ids[batch_idx, seq_idx] != -100:
                example_labels.append(index2tag[label_ids[
                                                  batch_idx][
                                                  seq_idx]])
                example_preds.append(index2tag[preds[
                                                  batch_idx][
                                                  seq_idx]])
        labels_list.append(example_labels)
        preds_list.append(example_preds)

    return preds_list, labels_list

def compute_metrics(eval_pred):
    y_pred, y_true = align_predictions(eval_pred.predictions,
                                       eval_pred.label_ids)

    f1 = f1_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    accuracy = accuracy_score(y_true, y_pred)

    return {'f1': f1,
            'precision': precision,
            'recall': recall,
            'accuracy': accuracy}
```

4.12 Training

We can now initialize a `Trainer` and pass the loaded model, the training arguments, the tokenizer, the data collator, the callback, and the encoded data splits for training and validation to it, and then start the training:

```
from transformers import Trainer

trainer = Trainer(model=model, args=training_args,
                                data_collator=data_collator,
                                compute_metrics=
                                compute_metrics, train_dataset
                                =ds_encoded['train'],
                                eval_dataset=ds_encoded['test'
                                ], tokenizer=tokenizer,
                                callbacks=[CALLBACK])

trainer.train()
```

We can also create a model card after training which is useful to store training parameters and metrics:

```
trainer.create_model_card()
```

4.13 Evaluation Results

The following section explores the results of fine-tuning the selected models on the two datasets separately, and on the final combined dataset.

Table 4.5 provides the baselines for orientation. Table 4.6 contains the results from fine-tuning on the Telegram dataset. Most of the achieved F1 scores surpass 0.7 and therefore outperform baseline no. 2, none of them gets close to baseline no. 1. The DistilBERT model achieved the lowest score (F1=0.6481), and the German-language BERT model by deepset and dbmdz performed best (F1=0.7482). We can furthermore observe that training was

stopped after a maximum of 3 epochs, sometimes already after 1 epoch. This tells us that the model's performance regarding the validation loss starts decrease rather soon. This is probably due to the rather small size of provided data, which tends to favor overfitting, meaning that the model learns patterns in the training data which are not helpful for improving its predictions on unseen data (cf. Jurafsky and Martin 2021c, p. 88). A similar behaviour could be observed for the other experiments.

Table 4.5: Baselines

No.	**Model**	**F1 score**
1	deepset/gbert-base (Chan, Schweter, and Möller 2020)	87.98
2	ousia (Yamada, Takeda, and Takefuji 2015)	56.41

Table 4.6: Fine-tuning different models on Telegram corpus

Model	**F1 score**	**Validation Loss**	**Epoch**
`distilbert-base-multilingual-cased`	0.6481	0.2910	1
`dbmdz/bert-base-german-uncased`	0.7440	0.2285	2
`deepset/gbert-base`	**0.7482**	0.2524	2
`xlm-roberta-base`	0.7330	**0.2459**	3
`cardiffnlp/twitter-xlm-roberta-base`	0.7103	0.2648	3

The results shown in table 4.7 are a bit more promising. We can see that the tested models overall performed better on the adjusted Smart-data corpus, with most F1-scores exceeding F1=0.8. The German-language BERT model by dbmdz is an exception with the lowest score out of all results (0.6156), however still outperforming baseline no. 2. Once again, the German-language BERT model by deepset and dbmdz performed best with an F1-score of 0.8535. Even though this is still below baseline no. 1, the achieved score seems promising, particularly when considering that baseline no. 1 was achieved using a larger corpus for fine-tuning than provided here.

Table 4.7: Fine-tuning different models on adjusted Smartdata corpus

Model	F1 score	Validation Loss	Epoch
distilbert-base-multilingual-cased	0.8203	0.1456	2
dbmdz/bert-base-german-uncased	0.6156	0.2560	3
deepset/gbert-base	**0.8535**	**0.1114**	3
xlm-roberta-base	0.8264	0.1445	1
cardiffnlp/twitter-xlm-roberta-base	0.8266	0.1344	3

Table 4.8 shows the results from fine-tuning on the combined dataset. We can observe that all models outperform baseline no. 2, while none of them is able to compete with baseline no. 1. Furthermore, we observe that the scores improved compared to fine-tuning only on the Telegram corpus. We can assume that this is due to the larger size of training data provided. However, compared to the fine-tuning on the adjusted Smartdata corpus only, the scores have decreased. It can be assumed that this might be party due to the underrepresentation of the ACTION class which is non-existent in the Smartdata corpus. Furthermore, inconsistencies in the annotation procedure (e.g. regarding hashtags) might also confuse the models.

Table 4.8: Fine-tuning different models on combined corpus

Model	F1 score	Validation Loss	Epoch
distilbert-base-multilingual-cased	0.6634	0.2543	2
dbmdz/bert-base-german-uncased	0.7775	0.1751	1
deepset/gbert-base	0.7922	**0.1751**	2
xlm-roberta-base	**0.7988**	0.1842	3
cardiffnlp/twitter-xlm-roberta-base	0.7768	0.2061	3

The best performing model regarding the F1-score in this experiment was the XLM-RoBERTa model (F1=0.7988), followed by the German-language BERT model by deepset and dbmdz. However, when comparing the validation loss of both models, the BERT model performs better than the XLM-RoBERTa model. Furthermore, a more detailed look into the precision and recall scores of both models reveals that the German-language BERT model achieves better precision than the RoBERTa model, as shown in 4.9. This

means that the results obtained from the deepset/gbert-base model can be considered more relevant than those of the xlm-roberta-base model.

Against this background, and taking into account that the deepset/gbert-base model performed best in both fine-tuning experiments with the separate datasets, the German BERT-model was selected as best model and used for application on real-world Telegram data in chapter 5.

Table 4.9: Detailed metrics for two best models

Model	F1 score	Precision	Recall	Validation Loss	Epoch
deepset/gbert-base	0.7922	**0.7816**	0.8032	**0.1751**	2
xlm-roberta-base	**0.7988**	0.7682	**0.8319**	0.1842	3

4.14 Further Evaluation and Error Analysis

For further evaluation, the following method can be used which returns a dataframe containing the relevant metrics per epoch:

```
def get_training_history(trainer):
    df = pd.DataFrame(trainer.state.log_history)[['epoch','
                                    loss' ,'eval_loss', '
                                    eval_f1', 'eval_precision'
                                    , 'eval_recall', '
                                    eval_accuracy']]
    df = df.rename(columns={"epoch":"epoch",
                            "loss": "training_loss",
                            "eval_loss": "validation_loss",
                            "eval_f1":"f1",
                            "eval_precision": "precision",
                           'eval_recall': 'recall',
                            'eval_accuracy': 'accuracy'})
    df['epoch'] = df["epoch"].apply(lambda x: round(x))
    df['training_loss'] = df["training_loss"].ffill()
    df[['validation_loss', 'f1']] = df[['validation_loss', '
                                    f1']].bfill().ffill()
```

```
    df.drop_duplicates()

    return df
```

We can visualize the results by creating some simple line plots with the following code:

```
eval_df = get_training_history(trainer)

# plot train, eval loss
sns.set_style("whitegrid")
ax = sns.lineplot(data = eval_df[['training_loss', '
                                  validation_loss']])
ax.set_xticks(range(len(eval_df)),labels=range(len(eval_df)))
plt.show()

# plot f1, precision, recall, accuracy
sns.set_style("whitegrid")
ax = sns.lineplot(data = eval_df[['f1', 'precision', 'recall'
                                  , 'accuracy']])
ax.set_xticks(range(len(eval_df)),labels=range(len(eval_df)))
plt.show()
```

Figure 4.3 and 4.4 show evaluation metrics for the fine-tuned deepset/gbert-base model. We can make the following observations:

The loss on the training set intersects with the validation loss between epoch 1 and 2. After that, the training loss continues to decrease, while the validation loss remains relatively constant, and then increases at epoch 5. This can be interpreted as overfitting, as we have already observed when looking at the number of epochs in the results above.

In the second plot we see that accuracy is continuously high with around 0.95. This is, however, not very informative for an NER model, since we are dealing with a very imbalanced dataset (see section **??**.

We can furthermore observe that recall for this model is in general higher than its precision. This means that the model tends to return less false

negatives than false positives. Its predictions are thus better in terms of completeness than in terms of relevance or validity.

```
from torch.nn.functional import cross_entropy

num_labels = 13

def forward_pass_with_label(batch):
    # Convert dict of lists to list of dicts suitable for
                                    data collator
    features = [dict(zip(batch, t)) for t in zip(*batch.
                                    values())]
    # Pad inputs and labels and put all tensors on device
    batch = data_collator(features)
    input_ids = batch["input_ids"].to(device)
    attention_mask = batch["attention_mask"].to(device)
    labels = batch["labels"].to(device)
    with torch.no_grad():
        # Pass data through model
        output = trainer.model(input_ids, attention_mask)
        # Logit.size: [batch_size, sequence_length, classes]
        # Predict class with largest logit value on classes
                                        axis
        predicted_label = torch.argmax(output.logits, axis=-1
                                        ).cpu().numpy()
```

Figure 4.3: Fine-tuning the deepset/gbert-base model: Training and validation loss

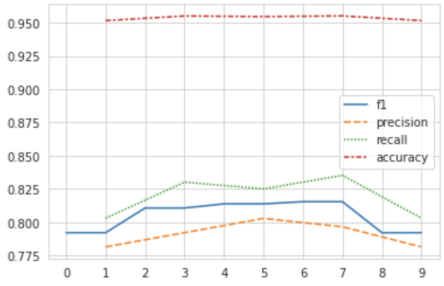

Figure 4.4: Fine-tuning the deepset/gbert-base model: F1, precision, recall, and accuracy

```
    # Calculate loss per token after flattening batch
                                    dimension with view
    loss = cross_entropy(output.logits.view(-1, num_labels),
                         labels.view(-1), reduction="none")
    # Unflatten batch dimension and convert to numpy array
    loss = loss.view(len(input_ids), -1).cpu().numpy()

    return {"loss":loss, "predicted_label": predicted_label}

valid_set_gbert = eval_encoded
valid_set_gbert = valid_set_gbert.map(forward_pass_with_label
                                , batched=True, batch_size=32)
df = valid_set_gbert.to_pandas()
```

We can now further work with this dataframe to explore the loss on token and class level, and plot a confusion matrix.[10]

Token-level errors

	0	1	2	3	4	5	6	7	8	9
input_tokens	.	-	Uhr	,	#	)	(	:	@	/
count	647	351	64	402	110	110	112	262	150	92
mean	0.17	0.27	0.75	0.11	0.4	0.38	0.3	0.11	0.18	0.24
sum	112.12	94.77	47.83	44.79	44.51	42.12	33.42	28.75	26.45	21.82

Figure 4.5: Tokens ranked by mean loss, German-language BERT model

Observations . token has highest total loss, not surprising since most common token in the list, and also part of named entities (count: 647), whitespace does not appear here (anders als bei roberta)

[10] For implementation details see `https://github.com/eli-quereli/ner-german-telegram/blob/main/src/notebooks/4.2_training/02_training/model_experiments_combined_datasets/gbert_smartdata_telegram.ipynb`

tokens: - # : Uhr ( ) , / are rarer, but have relatively high mean loss -¿ appear often together with named entities, are even sometimes part of them, that's why model mixes them up; probably need to revise annotations / annotation guide

Class-level erros

	0	1	2	3	4	5	6	7	8	9	10	11	12
labels	I-ACTION	B-ACTION	I-ORG	I-LOC	B-DATE	B-ORG	B-TIME	I-DATE	I-PER	I-TIME	B-PER	B-LOC	O
count	6	47	251	315	214	416	95	166	127	61	142	852	12862
mean	5.62	1.21	1.05	0.85	0.8	0.78	0.67	0.52	0.51	0.48	0.4	0.27	0.06
sum	33.71	56.65	263.46	267.29	171.45	324.4	63.18	86.67	64.89	29.38	57.11	228.98	821.87

Figure 4.6: Classes ranked by mean loss, German-language BERT model

- I-ACTION has highest loss, ua because very rare (only 6 occurences) -¿ need for more data - same for B-ACTION (same with roberta-model, label is problematic, maybe drop or label more) - I-ORG, I-LOC? - lowest loss: O - lowest loss labels: B-LOC, B-PER, I-TIME, I-PER, I-DATE

Confusion matrix

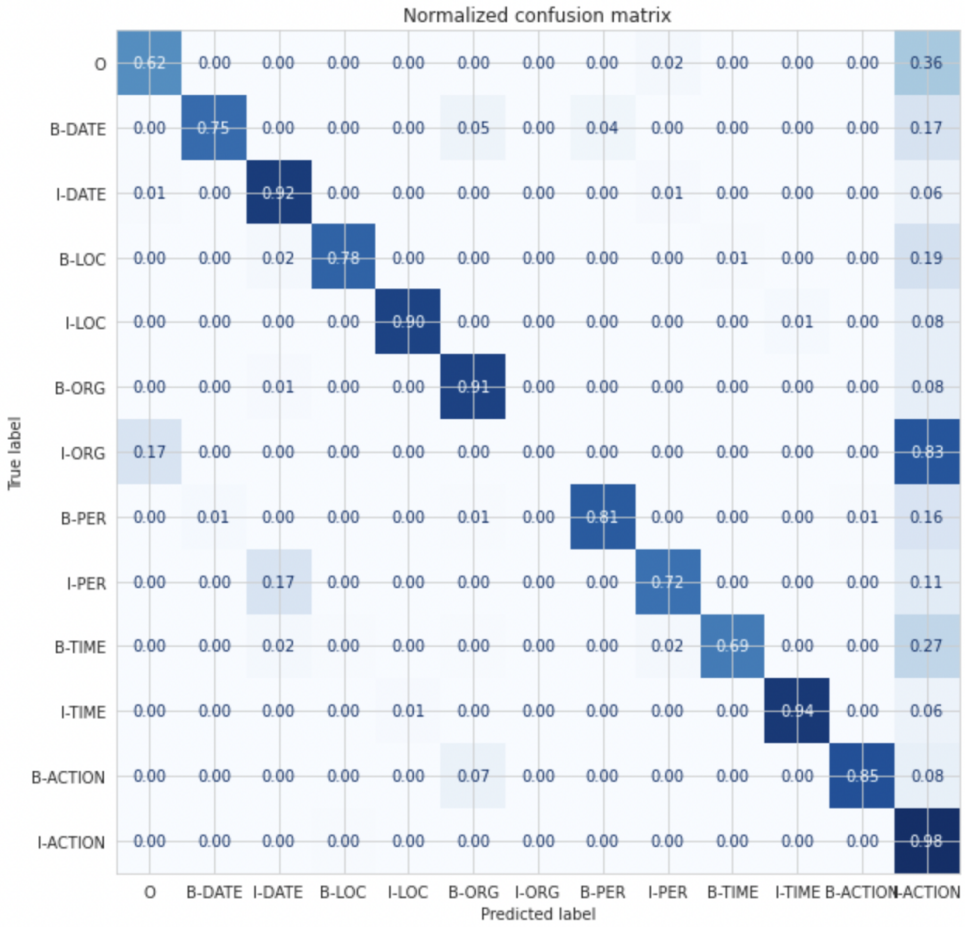

Figure 4.7: Confusion matrix, German-language BERT model

4.14.1 Post-processing

The values we get as output from our model don't necessarily make sense by themselves

Those are not probabilities but logits, the raw, unnormalized scores outputted by the last layer of the model. To be converted to probabilities, they need to go through a SoftMax layer (all Transformers models output the

logits, as the loss function for training will generally fuse the last activation function, such as SoftMax, with the actual loss function, such as cross entropy)

To get the labels corresponding to each position, we can inspect the id2label attribute of the model config

4.15 Summarization

Chapter 5

Application: Extracting Named Entities from a Telegram Channel

5.0.1 Example messages

Observations: model now detects time and date to some extent, did not recognize action Demo-Berlin, but unlike Flair model does not tag is as a location

makes errors, e.g. postcal code missing last digit, 0 as time entitity, Kurpfalz as location instead of offene gesellschaft kurpfalz

Table 5.1 shows the results of the German-language Flair model and the custom fine-tuned model for the example texts:

5.0.2 Larger sample

Table 5.1: Detected entities for German-language example messages: Flair vs. Custom model

No.	Text	flair/ner-german	gbert-base/finetuned
1	Freiburg Aufzug mit Endkundgebung am Platz der alten Synagoge . Live Musik ! Details 19062021 Raus auf die Straße	Freiburg[LOC]	Freiburg[LOC]
2	GEHT DEMONSTRIEREN ! - Message von Elijah Tee an die Dresdner & anderen Demonstranten in Deutschland Morgen am 13.06. um 16.30 Uhr und bundesweit überall , wo es Mitdenker gibt . - Bilder und Videos hochladen - Diskussion Fahren oder Mitfahren	Elijah Tee[PER], Deutschland[LOC]	Elijah Tee[PER], Dresdner[LOC], Deutschland[LOC], 13.06.[DATE], 16.30 Uhr[TIME]
3	15831 Blankenfelde Mahlow Berliner Speckgürtel Ausfahrt Lichtenrade am 14.03.2021 14032021 Raus auf die Straße	Blankenfelde Mahlow[LOC], Lichtenrade[LOC]	1583[LOC], Blankenfelde[LOC], Mahlow[LOC], Berliner Speckgürtel[LOC], Ausfahrt[LOC], Lichtenrade[LOC], 14.03.2021[DATE]
4	2021 10 11 Sven Liebich fragt Ministerpräsident Haseloff nach dessen Verständnis von Versammlungsfre - Sven Liebich 0:23 Raus auf die Straßen Übersicht / Overview	Sven Liebich[PER, Haseloff[PER], Sven Liebich[PER]	20211011[DATE], Sven Liebich[PER], Haseloff[PER], Sven Liebich[PER], 0[TIME]
5	FÜR ALLE DEMOKRATEN GEFAHR : Die Polizei blockiert den Stern ! Deswegen laufen die Menschen vom Ernst-Reuther-Platz nach Südosten zum Breitscheitplatz und dort weiter auf den Kudamm ! ! Schließt euch an : Neue Kantstraße direkt Richtung Osten auf den Breitscheitplatz ! Von der Straße des 17. Juni Richtung Süden zum Kudamm !	FU[ORG], Ernst-Reuther-Platz[LOC], Breitscheitplatz[LOC], Kudamm[LOC], Kantstraße[LOC], Breitscheitplatz[LOC], Kudamm[LOC]	Ernst- Reuther-Platz[LOC], Breitscheitplatz[LOC], Kudamm[LOC], Neue Kantstraße[LOC], Breitscheitplatz[LOC], 17.Juni[DATE], Kudamm[LOC]
6	Dr. Gunter Frank : Wie Moralismus dazu führt das man Feinde sieht und vernichten will — Antihygienista - Offene Gesellschaft Kurpfalz 15:04 Raus auf die Straßen Übersicht / Overview	Gunter Frank[PER], Antihygienista[ORG], Offene Gesellschaft Kurpfalz[ORG]	Gunter Frank[PER], Antigienista- Offene Gesellschaft[ORG], Kurpfalz[LOC], 15:04[TIME]
7	Einfach nur Krank solche Politiker ! Was sollen wir mit ihm machen wenn unsere Kinder anhand dieser Impfung nen schaden erleiden ? Wer ist dann dafür verantwortlich , wen schmeißen wir dann in den Knast und nehmen sein ganzes Geld weg , sollen wir das mit dir machen ? Raus auf die Straße	*no entities detected*	*no entities detected*
8	27.11.2021 Frankfurt Durchsagen von gemischt mit Musik . Super Idee Für mehr Berichte folgen auf ... Raus auf die Straßen Übersicht / Overview	Frankfurt[LOC]	27.11.2021[DATE], Frankfurt[LOC]
9	Demo-Berlin : Der Mann mit dem Schild wurde soeben von der Polizei festgenommen . Circa 7 Einsatzwägen sind vor Ort und umzäunen den Park großräumig . Am Humboldthain gibt es nichts mehr zu gewinnen . # b2908 Raus auf die Straßen	Demo-Berlin[LOC], Humboldthain[LOC]	Berlin[LOC], Humboldthain[LOC]
10	Anthony Fauci : Definition von " vollständig geimpft " könnte geändert werden — Epoch Times Nachrichten 6:59 Raus auf die Straßen Übersicht / Overview	Anthony Fauci[PER], Epoch Times Nachrichten[ORG]	Anthony Fauci[PER], Epoch Times Nachrichten[ORG], 6:59[TIME]

Chapter 6

Conclusion

6.1 Summarization

6.2 Discussion

more data

improved annotation scheme (zb ( ) / nicht taggen, # splitten), wie umgehen mit hashtags für Demos zB #b2908

entity disambiguation

relationship tagging

future work -¿ main adaptation on large telegram corpus

Bibliography

Aggarwal, Charu C. (2018). *Machine Learning for Text*. en. Cham: Springer International Publishing. ISBN: 978-3-319-73530-6 978-3-319-73531-3. DOI: 10.1007/978-3-319-73531-3. URL: http://link.springer.com/10.1007/978-3-319-73531-3 (visited on 05/14/2022).

Akbik, Alan, Blythe, Duncan, and Vollgraf, Roland (2018). "Contextual String Embeddings for Sequence Labeling". In: *COLING 2018, 27th International Conference on Computational Linguistics*, pp. 1638–1649.

— (2022a). *flair/ner-english · Hugging Face*. (Accessed on 07/23/2022). URL: https://huggingface.co/flair/ner-english.

— (2022b). *flair/ner-german · Hugging Face*. (Accessed on 07/23/2022). URL: https://huggingface.co/flair/ner-german.

Akbik, Alan et al. (2019). "FLAIR: An easy-to-use framework for state-of-the-art NLP". In: *NAACL 2019, 2019 Annual Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, pp. 54–59.

Baldwin, Timothy et al. (July 2015). "Shared Tasks of the 2015 Workshop on Noisy User-generated Text: Twitter Lexical Normalization and Named Entity Recognition". In: *Proceedings of the Workshop on Noisy User-generated Text*. Beijing, China: Association for Computational Linguistics, pp. 126–135. DOI: 10.18653/v1/W15-4319. URL: https://aclanthology.org/W15-4319 (visited on 05/16/2022).

Barbieri, Francesco, Anke, Luis Espinosa, and Camacho-Collados, Jose (2021). *XLM-T: Multilingual Language Models in Twitter for Sentiment Analysis and Beyond*. DOI: 10.48550/ARXIV.2104.12250. URL: https://arxiv.org/abs/2104.12250.

Benikova, Darina, Biemann, Chris, and Reznicek, Marc (May 2014). "NoSta-D Named Entity Annotation for German: Guidelines and Dataset". In: *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14)*. Reykjavik, Iceland: European Language Resources Association (ELRA), pp. 2524–2531. URL: http://www.lrec-conf.org/proceedings/lrec2014/pdf/276_Paper.pdf.

Benikova, Darina et al. (2014). "Germeval 2014 named entity recognition: Companion paper". In: *Proceedings of the KONVENS GermEval Shared Task on Named Entity Recognition, Hildesheim, Germany*, pp. 104–112.

Chan, Branden, Schweter, Stefan, and Möller, Timo (2020). *German's Next Language Model*. DOI: 10.48550/ARXIV.2010.10906. URL: https://arxiv.org/abs/2010.10906.

Chinchor, Nancy (Sept. 1997). *MUC-7 Named Entity Task Definition*. (Accessed on 07/23/2022). URL: https://www-nlpir.nist.gov/related_projects/muc/proceedings/ne_task.html.

Chinchor, Nancy and Sundheim, Beth (Aug. 2003). *Message Understanding Conference (MUC) 6*. Artwork Size: 10240 KB Pages: 10240 KB Type: dataset. DOI: 10.35111/WBCC-Y063. URL: https://catalog.ldc.upenn.edu/LDC2003T13 (visited on 06/27/2022).

Collobert, Ronan and Weston, Jason (2008). "A Unified Architecture for Natural Language Processing: Deep Neural Networks with Multitask Learning". In: *Proceedings of the 25th International Conference on Machine Learning*. ICML '08. Helsinki, Finland: Association for Computing Machinery, 160–167. ISBN: 9781605582054. DOI: 10.1145/1390156.1390177. URL: https://doi.org/10.1145/1390156.1390177.

Collobert, Ronan et al. (2011). "Natural Language Processing (Almost) from Scratch". en. In: *NATURAL LANGUAGE PROCESSING*, p. 45.

Conneau, Alexis et al. (2019). "Unsupervised Cross-lingual Representation Learning at Scale". In: *CoRR* abs/1911.02116. arXiv: 1911.02116. URL: http://arxiv.org/abs/1911.02116.

Derczynski, Leon, Bontcheva, Kalina, and Roberts, Ian (Dec. 2016). "Broad Twitter Corpus: A Diverse Named Entity Recognition Resource". In: *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*. Osaka, Japan: The COLING 2016 Organizing Committee, pp. 1169–1179. URL: https://aclanthology.org/C16-1111.

Derczynski, Leon et al. (Mar. 2015). "Analysis of named entity recognition and linking for tweets". en. In: *Information Processing & Management* 51.2, pp. 32–49. ISSN: 03064573. DOI: 10.1016/j.ipm.2014.10.006. URL: https://linkinghub.elsevier.com/retrieve/pii/S0306457314001034 (visited on 06/27/2022).

Derczynski, Leon et al. (Sept. 2017). "Results of the WNUT2017 Shared Task on Novel and Emerging Entity Recognition". In: *Proceedings of the 3rd Workshop on Noisy User-generated Text*. Copenhagen, Denmark: Association for Computational Linguistics, pp. 140–147. DOI: 10.18653/v1/W17-4418. URL: https://aclanthology.org/W17-4418 (visited on 07/21/2022).

Devlin, Jacob et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. DOI: 10.48550/ARXIV.1810.04805. URL: https://arxiv.org/abs/1810.04805.

Ethnologue (May 2022). *How many languages are there in the world?* en. URL: https://www.ethnologue.com/guides/how-many-languages (visited on 06/19/2022).

ExplosionAI (2022a). *English · spaCy Models Documentation*. (Accessed on 07/23/2022). URL: https://spacy.io/models/en#en_core_web_md.

ExplosionAI (2022b). *EntityRecognizer · spaCy API Documentation*. (Accessed on 07/23/2022). URL: https://spacy.io/api/entityrecognizer.

— (2022c). *Facts & Figures · spaCy Usage Documentation*. en. URL: https://spacy.io/usage/facts-figures (visited on 07/23/2022).

— (2022d). *German · spaCy Models Documentation*. (Accessed on 07/23/2022). URL: https://spacy.io/models/de#de_core_news_md.

— (2022e). *TransitionBasedParser · spaCy API Documentation*. (Accessed on 07/23/2022). URL: https://spacy.io/api/architectures#TransitionBasedParser.

Grishman, Ralph and Sundheim, Beth (1996). "Message Understanding Conference-6: A Brief History". In: *COLING 1996 Volume 1: The 16th International Conference on Computational Linguistics*. URL: https://aclanthology.org/C96-1079 (visited on 06/27/2022).

Huang, Zhiheng, Xu, Wei, and Yu, Kai (2015). *Bidirectional LSTM-CRF Models for Sequence Tagging*. DOI: 10.48550/ARXIV.1508.01991. URL: https://arxiv.org/abs/1508.01991.

HuggingFace (2022a). *Glossary: Attention mask*. (Accessed on 08/05/2022). URL: https://huggingface.co/docs/transformers/glossary#attention-mask.

— (2022b). *Glossary: Token Type IDs*. (Accessed on 08/05/2022). URL: https://huggingface.co/docs/transformers/glossary#token-type-ids.

— (2022c). *Padding and truncation*. (Accessed on 08/04/2022). URL: https://huggingface.co/docs/transformers/pad_truncation.

— (2022d). *Summary of the tokenizers*. (Accessed on 08/04/2022). URL: https://huggingface.co/docs/transformers/tokenizer_summary.

Jurafsky, Daniel and Martin, James H (2021a). "Speech and Language Processing. An Introduction to Natura Language Processing, Computational Linguistics, and Speech Recognition". In: Third edition draft. Chap. Naive Bayes and Sentiment Classification. URL: https://web.stanford.edu/~jurafsky/slp3/ed3book_jan122022.pdf.

Jurafsky, Daniel and Martin, James H. (2021b). *Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. URL: https://web.stanford.edu/~jurafsky/slp3/ed3book_jan122022.pdf.

Jurafsky, Daniel and Martin, James H (2021c). "Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition". In: Third edition draft. Chap. Logistic Regression. URL: https://web.stanford.edu/~jurafsky/slp3/ed3book_jan122022.pdf.

— (2021d). "Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition". In: Third edition draft. Chap. Vector Semantics and Embeddings. URL: https://web.stanford.edu/~jurafsky/slp3/ed3book_jan122022.pdf.

— (2021e). "Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition". In: Third edition draft. Chap. Neural Networks and Neural Language Models. URL: https://web.stanford.edu/~jurafsky/slp3/ed3book_jan122022.pdf.

— (2021f). "Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition". In: Third edition draft. Chap. Sequence Labeling for Parts of Speech and Named Entities. URL: https://web.stanford.edu/~jurafsky/slp3/ed3book_jan122022.pdf.

— (2021g). "Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition". In: Third edition draft. Chap. Deep Learning Architectures for Sequence Processing. URL: https://web.stanford.edu/~jurafsky/slp3/ed3book_jan122022.pdf.

Koppert-Anisimova, Inara (2022). *Stay away from overfitting: L2-norm Regularization, Weight Decay and L1-norm Regularization techniques — by Inara Koppert-Anisimova — unpack — Medium*. (Accessed on 08/05/2022). URL: https://medium.com/unpackai/stay-away-from-overfitting-l2-norm-regularization-weight-decay-and-l1-norm-regularization-795bbc5cf958.

LDC (2005). *ACE (Automatic Content Extraction) English Annotation Guidelines for Entities version 6.6*. (Accessed on 07/23/2022). URL: https://www.ldc.upenn.edu/sites/www.ldc.upenn.edu/files/english-entities-guidelines-v6.6.pdf.

Magueresse, Alexandre, Carles, Vincent, and Heetderks, Evan (June 2020). *Low-resource Languages: A Review of Past Work and Future Challenges*. Number: arXiv:2006.07264 arXiv:2006.07264 [cs]. URL: http://arxiv.org/abs/2006.07264 (visited on 06/19/2022).

Mikolov, Tomas et al. (2013). *Efficient Estimation of Word Representations in Vector Space*. DOI: 10.48550/ARXIV.1301.3781. URL: https://arxiv.org/abs/1301.3781.

Nadeau, David and Sekine, Satoshi (Aug. 2007). "A survey of named entity recognition and classification". en. In: *Lingvisticae Investigationes* 30.1, pp. 3–26. ISSN: 0378-4169, 1569-9927. DOI: 10.1075/li.30.1.03nad. URL: http://www.jbe-platform.com/content/journals/10.1075/li.30.1.03nad (visited on 07/04/2022).

Nothman, Joel et al. (2013). "Learning multilingual named entity recognition from Wikipedia". In: *Artificial Intelligence* 194, pp. 151–175. ISSN: 0004-3702. DOI: https://doi.org/10.1016/j.artint.2012.03.006. URL: https://www.sciencedirect.com/science/article/pii/S0004370212000276.

Pennington, Jeffrey, Socher, Richard, and Manning, Christopher D. (2014). "GloVe: Global Vectors for Word Representation". In: *Empirical Methods*

in Natural Language Processing (EMNLP), pp. 1532–1543. URL: http://www.aclweb.org/anthology/D14-1162.

Ralph Weischedel Martha Palmer, Mitchell Marcus Eduard Hovy Sameer Pradhan Lance Ramshaw Nianwen Xue Ann Taylor Jeff Kaufman Michelle Franchini Mohammed El-Bachouti Robert Belvin Ann Houston (Oct. 2013). *OntoNotes Release 5.0 - Linguistic Data Consortium*. (Accessed on 07/23/2022). URL: https://catalog.ldc.upenn.edu/LDC2013T19.

Ramshaw, Lance and Marcus, Mitch (1995). "Text Chunking using Transformation-Based Learning". In: *Third Workshop on Very Large Corpora*. URL: https://aclanthology.org/W95-0107.

Ritter, Alan et al. (2011). "Named Entity Recognition in Tweets: An Experimental Study". In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. EMNLP '11. Edinburgh, United Kingdom: Association for Computational Linguistics, 1524–1534. ISBN: 9781937284114.

Schiersch, Martin et al. (2018). "A German Corpus for Fine-Grained Named Entity Recognition and Relation Extraction of Traffic and Industry Events". In: *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*. Ed. by Nicoletta Calzolari (Conference chair) et al. Miyazaki, Japan: European Language Resources Association (ELRA). ISBN: 979-10-95546-00-9.

Schweter, Stefan and Akbik, Alan (2020). *FLERT: Document-Level Features for Named Entity Recognition*. arXiv: 2011.06993 [cs.CL].

Schön, Saskia et al. (2018). *DFKI Annotation Guidelines, Version 1.0*. (Accessed on 07/23/2022). URL: https://github.com/DFKI-NLP/smartdata-corpus/blob/master/SmartData_Corpus_Annotation_Guidelines_Feb_2018_v1.0.pdf.

Singh, Anil Kumar (2008). "Natural Language Processing for Less Privileged Languages: Where do we come from? Where are we going?" In: *Proceed-*

ings of the IJCNLP-08 Workshop on NLP for Less Privileged Languages. URL: https://aclanthology.org/I08-3004 (visited on 06/19/2022).

Strauss, Benjamin et al. (Dec. 2016). "Results of the WNUT16 Named Entity Recognition Shared Task". In: *Proceedings of the 2nd Workshop on Noisy User-generated Text (WNUT)*. Osaka, Japan: The COLING 2016 Organizing Committee, pp. 138–144. URL: https://aclanthology.org/W16-3919 (visited on 07/21/2022).

Tjong Kim Sang, Erik F. (2002). "Introduction to the CoNLL-2002 Shared Task: Language-Independent Named Entity Recognition". In: *COLING-02: The 6th Conference on Natural Language Learning 2002 (CoNLL-2002)*. URL: https://aclanthology.org/W02-2024.

Tjong Kim Sang, Erik F. and De Meulder, Fien (2003). "Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition". In: *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pp. 142–147. URL: https://aclanthology.org/W03-0419 (visited on 06/19/2022).

Tunstall, Lewis et al. (2022). *Natural language processing with Transformers: building language applications with Hugging Face*. eng. First edition. Beijing Boston Farnham Sebastopol Tokyo: O'Reilly. ISBN: 978-1-09-810324-8.

Vajjala, Sowmya et al. (2020). *Practical natural language processing: a comprehensive guide to building real-world NLP systems*. First edition. OCLC: on1125266646. Sebastopol, CA: O'Reilly Media. ISBN: 978-1-4920-5405-4.

Vaswani, Ashish et al. (2017). *Attention Is All You Need*. DOI: 10.48550/ARXIV.1706.03762. URL: https://arxiv.org/abs/1706.03762.

Weichbrodt, Gregor and Stanjek, Grischa (2022). *Teledash – analysis and research software for Telegram*. URL: https://github.com/democ-de/teledash.

Yadav, Vikas and Bethard, Steven (Oct. 2019). *A Survey on Recent Advances in Named Entity Recognition from Deep Learning models*. Tech.

rep. arXiv:1910.11470. arXiv:1910.11470 [cs] type: article. arXiv. URL: http://arxiv.org/abs/1910.11470 (visited on 05/14/2022).

Yamada, Ikuya, Takeda, Hideaki, and Takefuji, Yoshiyasu (July 2015). "Enhancing Named Entity Recognition in Twitter Messages Using Entity Linking". In: *Proceedings of the Workshop on Noisy User-generated Text*. Beijing, China: Association for Computational Linguistics, pp. 136–140. DOI: 10.18653/v1/W15-4320. URL: https://aclanthology.org/W15-4320.

Zong, Chengqing, Xia, Rui, and Zhang, Jiajun (2021). "Information Extraction". en. In: *Text Data Mining*. Singapore: Springer Singapore, pp. 227–283. ISBN: 9789811600999 9789811601002. DOI: 10.1007/978-981-16-0100-2_10. URL: https://link.springer.com/10.1007/978-981-16-0100-2_10 (visited on 05/14/2022).

Appendix

List of Figures

List of Tables

List of Acronyms

Annotation Guidelines

The following annotation guidelines are a combined adaptation of Benikova, Biemann, and Reznicek 2014, LDC 2005, Chinchor 1997, and Schön et al. 2018.

.1 Named Entity - Basic Definition

Names are denominators for a single unique object in the real world. Only full noun phrases are considered as named entities. Pronouns and all other phrases can be ignored. Phrases only partly containing names (e.g. 'deutschlandweit), or adjectives referring to entities ('euklidisch') can be ignored.
Only one entity type per token should be annotated. In case of multiple options (e.g. 'Charité' could be referred to as organization or as location, same for 'Landtag' or 'Bundestag'), the context of usage should be considered for the final classification.
Named entities can consist of more than one token. The inital token of an entity should be tagged with the B-label of the corresponding entity type. The following tokens belonging to the same entity should be labelled with the I-label of the same type. For
Combined named entities including different entity types (e.g. 'Wien-Demo') should be tagged separately if possible (Wien[LOC]-Demo[ACTION]), or else tagged as the dominant entity type (here: [ACTION]) to avoid nested entities.

Whitespace should not be labelled as part of an entity. Token separators such aus /, - or . should be included if they are inside an entity sequence, and excluded if they separate two different entities, or an entity and a non-entity token. Hashtags should be included in the labeled span.
Dates and times can be represented by numbers (31.10.2019, 16:00), and strings ('Morgen', 'Uhr').
Emoticons, user mentions and (parts of) urls should not be tagged as named entities.

.2 Definition of Different Entity Types

For annotation, the the 6 entity types PERSON, LOCATION, ORGANIZATION, DATE, TIME, and ACTION are used. The label set consists of B-tag and I-tag for each entity type, resulting in a total of 12 labels.

Table 1 shows the core definitions and examples for the annotated entity types.

.3 Specifications

.3.1 Handling Mixed-language Messages

Skip mixed-language in which the actual message is non-German language.

Examples:

- *We are angry and pissed off - Raus auf die Straße Demotermine!*
- *Brilliant numbers in Carlisle ! Raus auf die Straßen @Demotermine*
- *MASSIMA DIFFUSIONE http://t.me/stopdittatura/1930 Raus auf die Straßen @Demotermine*

Table 1: Definition of Entity types

Label	**Description**	**Examples**
PER	Named, single individual, limited to humans. Fictional persons can be included, as long as they're referred to by name.	Karl Lauterbach, Merkel, Harry Potter, Kretscher, Karl Hilz, Markus
LOC	Names of politically or geographically defined locations, including countries, provinces, regions, cities, districts, addresses including streets, squares, and postal codes, (named) buildings or other man-made structures, shopping centers, parking lots, or restaurants, cardinal points. No metaphorical locations.	Deutschland, Frankreich, #BW, Berlin, Bundestag, Restaurant Walliserkanne, Straße des 17. Juni, Neptunbrunnen, Alex, Nollendorfplatz, Norden, Süden, Hellersdorf
ORG	A named corporate, governmental, or other organizational entity: Organization entities can be corporations, agencies, institutions, newspapers, tv/radio channels, other media platforms, political parties, non-governmental organizations, and political groups with a formal or informal organizational structure.	ARD, Polizei Berlin, Youtube, Telegram, Amadeu Antonio Stiftung, Querdenken711, Antifa, Bundesregierung, Gesundheitsamt, Französische Polizei, Facebook, Google, Bill Gates Foundation, KSV LiLi, FC St. Pauli, SGD-Ultras, Bündnis21 Hessen, Polizei, Polizeipräsidium Pforzheim
DATE	Absolute or relative date expressions	23.09.2022, 20210930, 3108, Morgen, Heute, 2019, #Sa2011
TIME	Specific, absolute time expressions referring to times of day, and time codes.	13:10, 13:12 Uhr, 16 Uhr, 6:49
ACTION	A specific instance of an action or event undertaken to express political beliefs, e.g. demonstrations, vigils, motorcades, walks in public.	Kundgebung, Demo, Demonstration, Spaziergang, Aufzug, Aufzug RufderTrommeln

.3.2 Handling Incomprehensible Messages

Skip incomprehensible messages.

Example: *"Sterne-Restaurant Walliserkanne . Naudreck . Bersets faule Tricks . Uniarzt panikfrei . Herzmuskel . Neuer Beitrag #Walliserkanne Raus auf die Straßen Demotermine! Übersicht / Overview"*

.3.3 Handling Short Messages

Skip short messages which do not include at least two named entities.

Examples:

- *"AnwaelteFuerAufklaerung Demotermine"*
- *"Egal wo ihr diesen , oder die kommenden Samstage seid ! RheinCandleLight Demotermine"*

.3.4 Handling Punctuation

Punctutaion should only be included in the tag if they appear in the middle of sequence representing an entity.

Example: *Donnerstag[B-DATE] ,[I-DATE] 17.04.2020[I-DATE]*

Hashtags

If a hashtag is part of a named entity, it should be included in the tag.

Examples:

- #Berlin[B-ORG]
- #BW[B-ORG]

- #CH[B-ORG]
- #Hardy[B-PER] Groeneveld[I-PER]
- #Mutigmacher[B-ORG]

If a hashtag is part of a sequence combining two named entities, it should be included in the tag for the first entity.

Example: #be[B-LOC]2908[B-DATE]

.4 Detailed Annotation Instructions for Entity Types

.4.1 PERSON

Person names can consist of given name and last name, or only first / only last name. Titles (e.g. Dr.) and forms of address (Herr, Frau) should not be annotated.
Examples:

- *Scholz[B-PER]*
- *Carolin[B-PER] Matthie[I-PER]*
- *Merkel[B-PER]*
- *Anselm[B-PER] Lenz[I-PER]*
- *Karl[B-PER]*

.4.2 LOCATION

Do not label derived location words (e.g. *Kreuzberger Nächte* as location.

If a a sequence contains a number of tokens representing a location entity (e.g. an address), use the B-tag for the initial token and the I-tag for all succeeding tokens belonging to the same entity, including special characters such as , - . or /. The sequence should be long enough to create a representation of an entity which can be understood without the succeeding tokens.

Examples:

- *Straße[B-LOC] des[I-LOC] 17.[I-LOC] Juni[I-LOC]*
- *Bad[B-LOC] Oldesloe[I-LOC]*
- *Elbe[B-LOC] Elster[I-LOC]*
- *Bremen[B-LOC]-Vegesack[I-LOC]*
- *Lutherstadt[B-LOC] Wittenberg[I-LOC]*
- *Landeplatz[B-LOC] Grießkirchen[I-LOC]*
- *Google[B-LOC] center[I-LOC]*
- *Stuttgarter[B-LOC] Landtag[I-LOC]*
- NOTE: *Berlin[B-LOC] Neptunbrunnen[B-LOC]*

If the sequence contains a specification of the location, include the specification using the I-tags.

Examples:

- *Königstraße[B-LOC] Richtung[I-LOC] HBF[I-LOC]*
- *Bernauer[B-LOC] Str[I-LOC] Richtung[I-LOC] Süden[I-LOC]*

- *Marktoberdorf[B-LOC]-Kaufbeuren[I-LOC]-Neugablonz[I-LOC]*
- *Bremen[B-LOC]-Vegesack[I-LOC]*
- *D-67454[B-LOC] Haßloch[I-LOC]*

.4.3 ORGANIZATION

If a a sequence contains a number of tokens representing an organization entity, use the B-tag for the initial token and the I-tag for all succeeding tokens belonging to the same entity, including special characters such as , - . or /. The sequence should be long enough to create a representation of an entity which can be understood without the succeeding tokens.

Examples:

- *Bundesregierung[B-ORG]*
- *Gesundheitsamt[B-ORG]*
- *Französische[B-ORG] Polizei[I-ORG]*
- *Google[B-ORG]*
- *Bill[B-ORG] Gates[I-ORG] Foundation[I-ORG]*
- *KSV[B-ORG] LiLi[I-ORG]*
- *FC[B-ORG] St.[I-ORG] Pauli[I-ORG]*
- *Bündnis21[B-ORG] Hessen[I-ORG]*
- *Polizeipräsidium[B-ORG] Pforzheim[I-ORG]*

.4.4 TIME

Time expressions can include specific times of day and time codes (usually appearing after URLs. Sequences denoting a time span / duration should be annotated with the B-tag for the initial token and I-tags for the succeeding tokens.

Examples.

- *15:00[B-TIME]*
- *17:00[B-TIME] Uhr[I-TIME]*
- *www.example.com[O] 00:30[B-TIME]*
- *Mittag[B-DATE]*
- *15:30[B-TIME] -[I-TIME] 19:30[I-TIME]*

.4.5 DATE

Date expressions can be numeric values or strings. Sequences denoting a date spanshould be annotated with the B-tag for the initial token and I-tags for the succeeding tokens.

Examples:

- *2019[B-DATE]*
- *14.12.21[B-DATE]*
- *2.7.21[B-DATE]*
- *31. [B-DATE] JULI[I-DATE]*
- *01122020[B-DATE]*

- *Pfingstmontag[B-DATE]*
- *HEUTE[B-DATE]*
- *3108[B-DATE]*
- *201905[B-DATE]*
- *2019/03/05[B-DATE]*
- *28. [B-DATE]+ [I-DATE] 29.8. [I-DATE]*
- *17042021[B-DATE] –[I-DATE] 21042021[I-DATE]*

.4.6 ACTION

Actions represent unique real-world events or actions. They must not necessarily be named, but be identifiable as unique object in the context in which they are mentioned. Also, the type of action should be expressed.

Examples:

- *Angriff[B-ACTION]*
- *Demo-B*
- *Autokorso[B-ACTION]*
- *Offenes[B-ACTION] Treffen[I-ACTION]*
- *Aufzug[B-ACTION] RufderTrommeln[I-ACTION]*
- *Zug[B-ACTION]*
- *Plakate[B-ACTION]*
- *Infostände[B-ACTION]*

- *Informationstouren[B-ACTION]*
- *Anti[B-ACTION]-Corona[I-ACTION]-Proteste[I-ACTION]*
- *Protestzug[B-ACTION]*
- *Spaziergänge[B-ACTION]*
- *Querdenker[B-ORG]-Demo[B-ACTION]*
- *Korso[B-ACTION] Nord[I-ACTION]*
- *Gedenken[B-ACTION]*
- *Aktionswoche[B-ACTION]*
- *Wahlkampf[B-ACTION]*
- *Live-Stream[B-ACTION]*

Statutory Declaration

I herewith formally declare that I have written the submitted thesis independently. I did not use any outside support except for the quoted literature and other sources mentioned in the paper.
I clearly marked and separately listed all of the literature and all of the other sources which I employed when producing this academic work, either literally or in content.
I am aware that the violation of this regulation will lead to the failure of the thesis.

.....	.....
Name	Signature
.....	.....
Matriculation Number	Place and Date